Learning Solvability Boundaries in LLMs: Improving Reasoning, Detecting Unsolvability, and Calibrating Refusal

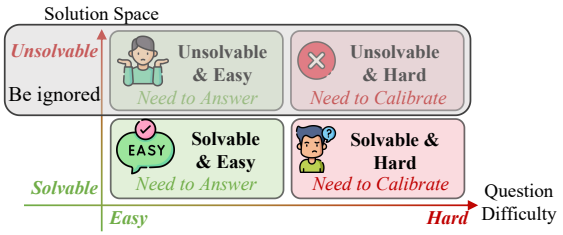
Anonymous ACL submission

Abstract

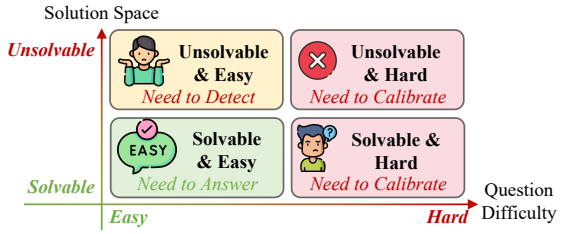
Reasoning alignment usually assumes each query has a determinate answer, rewarding models for reaching it and penalizing or ignoring failures. This assumption breaks on flawed queries with missing information or contradictory constraints, causing models to hallucinate invalid answers or over-refuse difficult valid tasks. We study this failure mode as *solvability boundary learning*: separating input-side answerability defects from model-side inability on well-posed tasks. To address this, we introduce **UnsolvableQA**, a verifier-backed multi-domain benchmark built with deterministic checks and audited math verification, and **UnsolvableRL**, an alignment framework that decouples reasoning, unsolvability detection, and capability-calibrated refusal. UnsolvableRL yields over 85% unsolvability detection on Qwen3-4B while improving solvable accuracy from 43.4% to 69.4%, with a +35.4-point solvable-accuracy gain on Llama3-8B. Diagnostic tuning further distinguishes missing-information defects from constraint conflicts and transfers to interactive clarification, improving Detail Recovery from 47.8% to 60.7% for Llama3-8B. Ablations reveal Capability Collapse: refusal penalties degrade detection without unsolvable data, but act as a rigor regularizer when such data are included.

1 Introduction

As Large Language Models (LLMs) achieve strong performance on complex reasoning tasks (Wei et al., 2022; Jaech et al., 2024; Hu et al., 2024), modern reasoning alignment often assumes that each query is well-posed and has a determinate answer (Chen et al., 2025c). Under this assumption, models can be rewarded for reaching the verified answer and penalized or left unrewarded otherwise (Shao et al., 2024; Guo et al., 2025). Yet many queries violate this premise: they may omit necessary information or contain contradictory constraints, so no determinate answer exists under the requested response format. Prior work on confidence calibration (Xiong et al., 2023; Yoon et al., 2025; Fisch et al., 2022; Chen et al., 2025b) and capability boundaries (Chen et al., 2024; Zhang et al., 2025; Kalai et al., 2025) mainly examines when models should abstain from unproductive reasoning on otherwise valid tasks; TruthRL (Wei et al., 2025) further encourages refusal under uncertainty via a ternary reward.



(a) Previous works about response based on Subjective Calibration.



(b) Response both based on Objective Detection and Subjective Calibration.

Figure 1: Comparison of alignment paradigms. We treat the solution space (solvable vs. unsolvable) and question difficulty (easy vs. hard) as orthogonal dimensions, requiring models to **detect input-side defects**, **solve feasible tasks**, and **calibrate refusal** on hard but solvable problems.

This model-side focus overlooks flawed queries whose specifications make a determinate answer impossible, such as missing information or logical contradictions. These cases are not merely difficult: the requested answer is not well-defined. We call the resulting distinction the *solvability boundary*, and treat *Solution Space* (solvable vs. unsolvable) and *Question Difficulty* (easy vs. hard) as orthogonal dimensions (Figure 1). A reliable system must

therefore decouple three behaviors: solving feasible tasks, detecting unsolvable inputs, and calibrating refusal when a solvable problem exceeds current capacity. Current LLMs often blur this boundary, leading to a "refusal dilemma": they hallucinate answers to invalid queries, or over-refuse valid but difficult problems.

To learn this boundary, we introduce **UnsolvableQA**, a benchmark suite of paired solvable and unsolvable instances across logic and mathematics, with science-domain diagnostic and OOD extensions, built via *Reverse Construction* with constraint conflicts and missing-information cases for diagnostic analysis. Verifier-backed structured domains use deterministic solvers; math instances undergo cross-model verification and human auditing. We further propose **UnsolvableRL**, optimizing (1) unsolvability detection, (2) a false-unsolvability penalty, and (3) dynamic capability calibration when empirical accuracy falls below a target threshold.

Empirical results show solvability boundary learning acts as an effective reasoning regularizer. UnsolvableRL improves the unsolvable detection rate of Qwen3-4B from 38.8% to 87.5%, while simultaneously increasing solvable problem accuracy from 43.4% to 69.4%. On Llama3-8B, mean score improves from 25.2% to 55.1% (+29.9pp; 119% relative improvement), with OOD gains on GPQA (+12.0%) and MMLU-Pro (+4.5%). Diagnostic tuning enables missing-information and constraint-conflict attribution and transfers to interactive clarification (Detail Recovery 47.8%→60.7% for Llama3-8B). Ablations reveal *Capability Collapse* when penalties are applied without unsolvable data, whereas our joint framework turns them into a rigor regularizer.

Overall, our main contributions are:

- We construct **UnsolvableQA**, a verifier-backed benchmark suite spanning logic and math, with science-domain diagnostic/OOD evaluation, built via Reverse Construction with deterministic checks and audited math verification.
- We propose **UnsolvableRL** for *solvability boundary learning*, decoupling reasoning on solvable tasks, unsolvability detection, and capability-calibrated refusal in a unified reinforcement alignment framework.
- We show that learning solvability boundaries improves both solvable accuracy and unsolvability detection, prevents capability collapse under refusal penalties, and extends beyond binary detection to defect diagnosis and zero-shot clarification transfer.

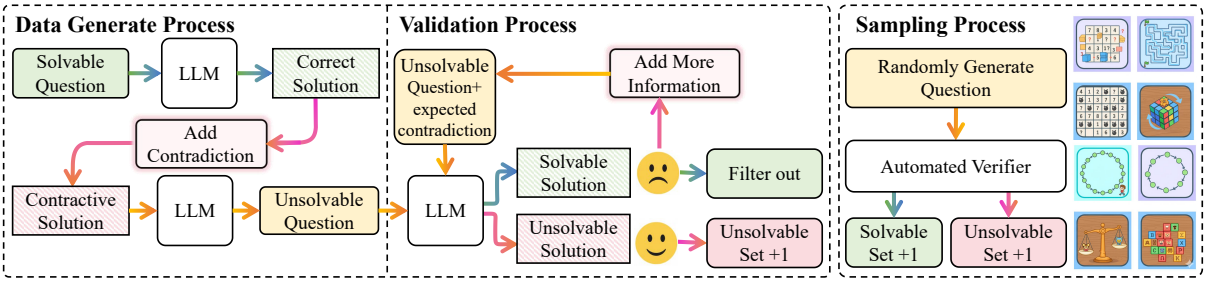
2 Related Work

Automated Reasoning Dataset Construction. Automated Dataset Construction (ADC) is essential for eliciting reasoning capabilities (Liu et al., 2024; Wang et al., 2023). While most methods synthesize solvable puzzles (Chen et al., 2025a, 2024) or verify rationale correctness (Ling et al., 2023; Chen et al., 2025c), research into unsolvability remains limited. SQuAD 2.0 (Rajpurkar et al., 2018) introduced unanswerable reading comprehension, while ReliableMath (Xue et al., 2025) recently established unsolvable mathematics benchmarks and utilized SFT/DPO to enhance model performance in identifying flawed problems.

Unanswerability Detection and Self-Awareness. Recent interpretability research identifies that unanswerability is linearly encoded within an LLM's activation space, enabling binary detection via specific directions (Lavi et al., 2025). Parallel studies on self-awareness and calibration (Zhang et al., 2025; Mei et al., 2025; Kalai et al., 2025; Steyvers et al., 2025) explore how models estimate subjective uncertainty and recognize knowledge boundaries to avoid hallucinations on difficult tasks.

Reinforcement Learning for Reasoning. RL is central to aligning LLMs with complex reasoning (Ouyang et al., 2022; Schulman et al., 2017; Rafailov et al., 2024). Algorithms like GRPO (Shao et al., 2024) and DAPO (Yu et al., 2025) significantly boost reasoning efficiency and capabilities (Sheng et al., 2025; Tu et al., 2025; Team et al., 2025; Guo et al., 2025; Wang et al., 2025; Chen et al., 2025b). To mitigate hallucinations, frameworks like TruthRL (Wei et al., 2025) apply penalties to encourage models to refuse answering under uncertainty.

Despite these advances, current paradigms have three gaps. First, existing datasets often rely on shallow linguistic perturbations rather than programmatically verifiable structural contradictions. Second, unsolvability detection remains a passive diagnostic tool for extractive tasks (Lavi et al., 2025), without active alignment strategies for identifying logical flaws. Finally, current RL frameworks conflate objective unsolvability with subjective uncertainty, which can trigger capability col-



(a) Mathematical Data Collection Methods. (b) Puzzle Data Collection Methods.

Figure 2: The **UnsolvableQA** data collection pipeline. **(a) Mathematical Data:** We inject constraint conflicts into the solution paths of solvable problems via "Reverse Construction," using LLM-based cross-verification and human auditing to validate the intended flaw. **(b) Puzzle Data:** Programmatic generators and deterministic verifiers (e.g., SAT solvers, DFS) rigorously classify generated instances into solvable and unsolvable sets.

lapse through undifferentiated refusal. We bridge these gaps by constructing inherently unsolvable data and learning a three-way decision boundary: solving feasible tasks, detecting input-side answerability defects, and refusing queries beyond the model's capacity.

3 UnsolvableQA Construction

Most existing benchmarks primarily focus on solvable tasks, which often leads to hallucinations during feasibility evaluations. To better understand this issue, we present UnsolvableQA using a dual-track methodology tailored to domain-specific verification (Figure 2).

3.1 Mathematical Data Collection

Programmatic verification is often infeasible for advanced math tasks. We therefore use **Reverse Construction** for the entire mathematical unsolvable-data pipeline (Figure 2a): starting from solvable seed problems and verified solution paths, we reverse-engineer input-side flaws that make the final answer undefined. The pipeline has two stages:

Data Generation Process. First, we filter seed problems, retaining only those where a strong LLM (DeepSeek-Reasoner) successfully generates a correct solution. Next, the same model plans and injects controlled constraint conflicts into the problem statement, such as mutually incompatible conditions or externally added constraints that make the original solution path invalid. Guided by the structural plan, these conflicts are precisely targeted within the problem statement while preserving the surface form of a plausible reasoning problem.

Validation Process. We further implement a multi-tier verification protocol. First, the generated problem is presented to DeepSeek-R1 for autonomous contradiction detection. If the model fails to identify the flaw independently, we provide it with the predefined contradiction plan to reassess solvability. To ensure logical rigor, we introduce a **Consensus-Based Cross-Verification** stage where Gemini 3 Pro evaluates the generated instance against the intended contradiction. If Gemini identifies loopholes, the two models engage in an iterative debate. An instance is retained only if both models reach a unanimous consensus on its unsolvability. Ultimately, three STEM Master's students manually audited the entire test set to guarantee data quality and logical consistency (detailed in Appendix A.10).

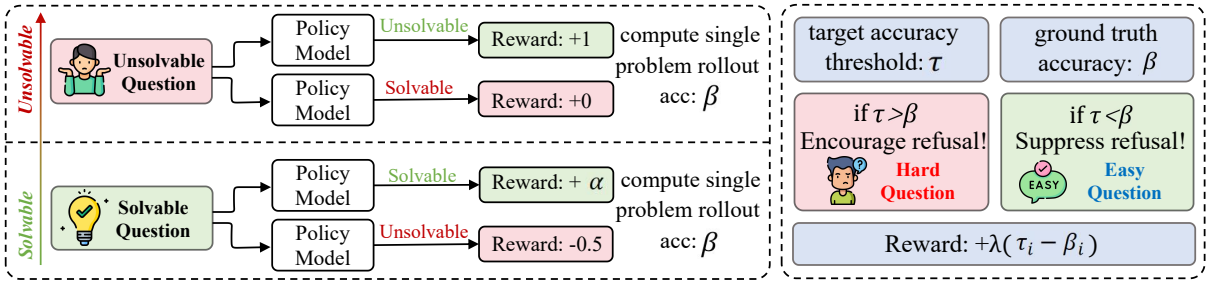
3.2 Puzzle Data Collection

We developed automated generators for four representative domains (Figure 2 (b)), employing deterministic solvers to guarantee strict unsolvability across all instances.

Game24. We enumerate all binary expression trees using exact rational arithmetic (Python Fraction) to avoid floating-point errors. Unsolvable instances are isolated via *rejection sampling* of number sets mathematically proven to lack solutions.

Hamiltonian Cycle/Path. Graph traversability is framed as a SAT problem and verified via pysat (MiniSat). We engineer unsolvable instances by injecting structural impossibilities, such as severing critical edges or creating bottlenecks.

Hitori. We model Hitori as a constraint satisfaction problem using python-constraint. Unsolvable variants are generated by injecting conflicting constraints, such as forcing duplicate numbers that violate fundamental adjacency rules.



(a) Reward for Unsolvability Detection (Objective). (b) Reward for Difficulty Calibration (Subjective).

Figure 3: UnsolvableRL reward design. The framework decouples three behaviors: reasoning accuracy on solvable instances, unsolvability detection for input-side defects, and capability calibration for solvable but currently hard instances. Panel (a) visualizes the objective solvability reward, including correctness reward on solvable queries, unsolvability reward on flawed queries, and a false unsolvability penalty for labeling solvable queries as unsolvable. Panel (b) visualizes capability calibration, which regulates `<beyond_capacity>` refusal by comparing the target threshold τ with the ground-truth-verified rollout accuracy β: refusal is encouraged when $\tau > \beta$ and discouraged when $\tau < \beta$.

Split	Domain	Solvable	Unsolvable	Total
Train	Game24	50	50	100
Train	Hamiltonian Cycle	48	48	96
Train	Hamiltonian Path	50	50	100
Train	Hitori	50	50	100
Train	Maze	100	59	159
Train	AIME (1983–2023)	50	32	82
Test	Game24	50	50	100
Test	Hamiltonian Cycle	48	50	98
Test	Hamiltonian Path	50	50	100
Test	Hitori	50	50	100
Test	Maze(Easy)	100	94	194
Test	Maze(Hard)	100	100	200
Test	AIME (2024–2025)	60	47	107
Train	Total	348	289	637
Test	Total	458	441	899

Table 1: Unified vertical statistics for **UnsolvableQA** train/test partitions. Train Maze has no difficulty split; AIME year ranges differ.

Maze Navigation. Base mazes are generated via randomized DFS. To ensure unsolvability, we obstruct a *critical junction* shared by all valid paths and verify the resulting impossibility via BFS.

3.3 Dataset Statistics and Quality Control

UnsolvableQA is domain-balanced (Table 1), comprising 899 test instances (458 solvable, 441 unsolvable) and 637 training instances (348 solvable, 289 unsolvable). Training prompts utilize templated formats with aggregated difficulties, whereas test prompts are diverse and preserve fine-grained splits, including disjoint AIME years[1]. For rigorous quality control, logic puzzles are verified via rule-based solvers, while math problems undergo multi-pass LLM validation and manual review. Detailed construction of the fine-grained diagnostic dataset, categorized by Missing Information (MI) and Constraint Conflict (CC), along with two additional out-of-distribution test sets, are provided in Appendix A.4 and A.6, respectively.

[1]AIME year ranges: train 1983–2023; test 2024–2025.

4 UnsolvableRL Methodology

The reward design is intended to prevent three behaviors from collapsing into a single refusal action: solving valid problems, detecting input-side defects, and abstaining from solvable but currently too-hard problems.

4.1 Problem Formulation

We model the task as generating a response y for instance $x \in \mathcal{D}$, where x is categorized by solvability ($\mathcal{S}$ or $\mathcal{U}$). Difficulty is policy-relative rather than an intrinsic ground-truth label: a solvable instance is treated as currently hard when the model's ground-truth-verified rollout accuracy falls below the dynamic target threshold. We optimize policy π_θ for three behavioral objectives: (1) **Reasoning Accuracy** for solvable instances the current policy can reliably answer, ensuring correct reasoning paths and answers; (2) **Unsolvability Detection** for $x \in \mathcal{U}$, requiring the model to output a specific tag (e.g., `<unsolvable>`) to avoid hallucinations on input-side defects; (3) **Capability Calibration** for solvable instances that are empirically hard for the current policy, enforcing prudent abstention (e.g., `<beyond_capacity>`) when verified rollout accuracy is below τ.

4.2 Group-Relative Policy Optimization

We adopt Group Relative Policy Optimization (GRPO) to optimize π_θ without a separate value function. For each x, the policy samples a group of G outputs $\mathcal{Y} = \{y_i\}_{i=1}^{G}$. The objective is:

$$\mathcal{J}(\theta) = \mathbb{E}_{x,\mathcal{Y}}\Bigg[\frac{1}{G}\sum_{i=1}^{G}\min\Big(r_i(\theta)A_i, \text{clip}(r_i(\theta), 1-\epsilon, 1+\epsilon)A_i\Big)\Bigg] \quad (1)$$

where $r_i(\theta) = \frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)}$. Advantage A_i is estimated by normalizing rewards within the group: $A_i = (R(y_i \mid x) - \mu_{\mathbf{R}})/(\sigma_{\mathbf{R}} + \epsilon)$, where $\mu_{\mathbf{R}}$ and $\sigma_{\mathbf{R}}$ are the mean and standard deviation of rewards $\mathbf{R} = \{R(y_j \mid x)\}_{j=1}^{G}$.

4.3 Reward Design

As Figure 3 illustrates, UnsolvableRL implements this decoupling via three orthogonal reward components that must not collapse into undifferentiated refusal: $R = R_{\text{acc}} + R_{\text{detect}} + R_{\text{cal}}$.

1) Reasoning Accuracy (R_{acc}): For **solvable instances**, the model receives $R_{\text{acc}} = \alpha \in \{0, 1\}$ based on deterministic verification (e.g., SAT solvers or exact matchers). Incorrect reasoning yields a neutral reward ($\alpha = 0$) rather than a penalty, preserving the incentive to attempt complex chains of thought.

2) Unsolvability Detection (R_{detect}): This reward targets input-side answerability defects, especially missing support and constraint conflicts that make the problem specification incomplete or inconsistent. On **unsolvable problems**, outputting the canonical `<unsolvable>` tag earns $+1$ reward; hallucinating an answer yields 0. Conversely, applying the `<unsolvable>` tag to a solvable problem incurs a *False Unsolvability Penalty* ($\rho = -0.5$). As analyzed in Appendix A.8, this penalty increases the cost of false unsolvability and discourages universal rejection.

3) Difficulty Calibration (R_{cal}): To regulate subjective refusal on valid problems, we introduce a *dynamic accuracy threshold* $\tau \in [0, 1]$. Let β be the empirical accuracy of the generation group's solvable subset; the calibration reward for outputting `<beyond_capacity>` is:

$$R_{\text{cal}}(y_i) = \lambda(\tau - \beta)\,\mathbf{1}[y_i \text{ contains } \texttt{<be..>}]. \quad (2)$$

This incentive promotes prudent abstention when current capabilities fall short ($\beta < \tau$) and penalizes lazy refusals when the model is empirically capable ($\beta > \tau$). We implement a *progressive schedule*, monotonically increasing $\tau \to 1$ during training to safely transition from exploration to strict accuracy constraints, preventing reward collapse (Appendix A.9).

5 Experiments

We trained and evaluated **Deepseek-Distill-Llama3-8B** (Llama3-8B) (Grattafiori et al., 2024; Guo et al., 2025) and **Qwen3-4B** (Yang et al., 2025) in think mode on UnsolvableQA, reporting AVG@8 for AIME partitions and AVG@4 elsewhere. In our standardized setup, the final Qwen3-4B and Llama3-8B models are trained for 320 and 400 steps, respectively, while the ablation models were trained for 120 steps due to computational cost constraints. We use the longer runs to report converged performance, and use the 120-step ablations only for controlled, fixed-budget component attribution; therefore, ablation conclusions are drawn within the 120-step setting.

5.1 Main Results

UnsolvableRL significantly enhances reasoning reliability across architectures. As shown in Table 2, applying our framework to Llama3-8B improves its mean reasoning score from 25.2% to **55.1%**. Similarly, Qwen3-4B achieves a competitive global mean of **78.6%**, substantially narrowing the gap with massive proprietary models such as GPT-5.2-Low and Gemini-3. Crucially, the 4B model achieves an unsolvable detection rate (U) of **87.5%** while simultaneously boosting its solvable accuracy (S) from 43.4% to **69.4%**.

Performance gains generalize effectively to external OOD benchmarks. Beyond in-domain evaluations, aligned models show consistent improvements on MMLU-Pro, GPQA, and the externally-constructed unsolvable mathematics benchmark ReliableMath, as well as two distinct OOD logic tasks (8-Puzzle and Zebra Logic) detailed in A.6. Notably, on **ReliableMath**, Llama3-8B achieves **68.3%** (a +12.1% gain), demonstrating strong competitiveness against much larger systems. Similarly, GPQA shows a **+12.0%** increase in identifying graduate-level unsolvable instances. Detailed breakdowns are provided in Appendices A.1 & A.6. Beyond binary boundary learning, we further evaluate fine-grained defect attribution and zero-shot clarification transfer (§5.2).

Dataset	Metric	Advanced LLMs			Llama3-8B		Qwen3-4B	
		Deepseek-V3.2-R	GPT-5.2-Low	Gemini-3	Distill	+ UnsolvableRL	Instruct	+ UnsolvableRL
Game24	S / U	**98.0** / 77.5	64.0 / 66.0	94.0 / 94.0	28.0 / 48.0	80.0 / 22.0	81.0 / 17.0	94.5 / 99.0
	Mean	87.8	65.0	94.0	38.0	51.0 (+13.0)	49.0	**96.8** (+47.8)
HamCycle	S / U	83.5 / 94.0	97.9 / 94.0	**100.0 / 98.0**	27.1 / 2.0	52.1 / 100.0	37.5 / 57.0	41.1 / 94.5
	Mean	88.8	96.0	**99.0**	14.6	76.1 (+61.5)	47.3	67.8 (+20.5)
HamPath	S / U	84.0 / 96.0	92.0 / 94.0	**100.0** / 90.0	14.0 / 0.0	74.0 / 92.0	37.0 / 56.5	55.0 / **96.5**
	Mean	90.0	93.0	**95.0**	7.0	83.0 (+76.0)	46.8	75.8 (+29.0)
Hitori	S / U	70.0 / 86.0	62.0 / 98.0	**98.0 / 100.0**	6.0 / 56.0	30.0 / 76.0	34.5 / 6.5	63.5 / 94.5
	Mean	78.0	80.0	**99.0**	31.0	53.0 (+22.0)	20.5	79.0 (+58.5)
Maze (Easy)	S / U	**100.0** / 98.9	98.0 / **100.0**	99.0 / 94.6	1.0 / 44.7	67.0 / 42.6	32.7 / 55.6	96.5 / 98.9
	Mean	**99.5**	99.0	96.8	22.9	54.8 (+31.9)	44.1	97.7 (+53.6)
Maze (Hard)	S / U	**98.0 / 98.0**	86.0 / 94.0	96.0 / 91.0	0.0 / 52.0	13.0 / 35.0	13.3 / 64.5	65.8 / 89.0
	Mean	**98.0**	90.0	93.5	26.0	24.0 (-2.0)	38.9	77.9 (+39.0)
AIME 24-25	S / U	85.0 / 40.3	90.0 / **42.5**	**95.0** / 21.2	35.0 / 39.4	43.3 / 44.7	67.9 / 14.8	69.6 / 40.4
	Mean	62.7	**66.3**	58.1	37.2	44.0 (+6.8)	41.4	55.0 (+13.6)
Overall	**S / U**	88.4 / 84.4	84.3 / 84.1	**97.4** / 84.1	15.9 / 34.6	51.3 (+35.4) / 58.9 (+24.3)	43.4 / 38.8	69.4 (+26.0) / **87.5** (+48.7)
	Mean	86.1	84.2	**90.8**	25.2	55.1 (+29.9)	41.1	78.6 (+37.5)
MMLU-Pro	Mean	85.0	87.5	**90.1**	52.4	56.9 (+4.5)	62.0	64.4 (+2.4)
GPQA	Mean	82.4	**92.4**	91.9	34.5	46.5 (+12.0)	54.1	54.5 (+0.4)
ReliableMath	Mean	67.0	50.4	58.5	56.2	**68.3** (+12.1)	41.6	62.4 (+20.8)

Table 2: Performance on reasoning and general knowledge benchmarks. Reasoning datasets report **Solvable Accuracy (S)**, **Unsolvable Detection Rate (U)**, and their **Mean (M)**. Parentheses denote absolute improvements over the base model; **global best scores** are bolded.

5.2 Fine-grained Diagnosis and Clarification Transfer

To move beyond binary detection, we extend our framework into a Diagnostic Reasoning stage. In structured reasoning domains, we instantiate two answerability-defect families: Missing Information (MI) and Constraint Conflict (CC). MI denotes missing support needed for a determinate answer under the requested response format, while CC captures mutually incompatible observed constraints that make the problem specification inconsistent.

To support this, we utilize seeds from AIME (1983) and the MMLU-Pro validation set to synthesize a diagnostic training set of 259 instances (120 solvable and 139 unsolvable), covering only missing-information and constraint-conflict families. For rigorous evaluation, we curate a separate diagnostic OOD test set comprising 320 instances from entirely disjoint sources: 149 diagnostic instances from AIME (2024–2025), distinct from the core AIME split in Table 1 and including additional MI cases for attribution evaluation, to evaluate temporal generalization; and 171 instances from GPQA (denoted as GPQA-U) to assess cross-domain transferability in graduate-level science. Models were initialized from converged UnsolvableRL models and underwent an additional 80-step diagnostic fine-tuning phase on the synthesized training set to output specific tags for diagnostic purposes.

The transition from binary unsolvability detection to cause-aware diagnosis yields three insights:

Metrics / Datasets	Qwen3-4B		Llama3-8B	
	Instruct	+ Diag.	Distill	+ Diag.
Reasoning Accuracy (Solvable S ↑)				
MMLU-Pro	62.0	65.4	52.4	54.8
GPQA	54.1	55.8	34.5	50.0
AIME	67.9	69.6	35.0	48.8
Unsolvability Detection (Unsolvable U ↑)				
GPQA-U	21.7	33.8	21.5	36.6
AIME-MI	23.4	41.4	38.3	38.8
AIME-CC	45.2	69.6	45.2	75.6
Interactive Transfer (IN3 Benchmark)				
Detail Recovery	60.5	66.5	47.8	60.7
Intention Coverage	34.7	39.7	20.4	23.0
Avg. Rounds	4.24	4.58	3.10	4.16

Table 3: Results for diagnostic reasoning and clarification transfer. +Diag. models show performance synergies and superior zero-shot transfer to interactive tasks (IN3), enabling proactive questioning.

Cause-aware diagnosis improves general reasoning. As shown in Table 3, training models to identify why a query is unsolvable yields additional gains beyond yes/no detection. Notably, diagnostic tuning improves Llama3-8B's GPQA accuracy from 34.5% to 50.0% (+15.5%), suggesting that learning the causes of invalidity can regularize valid reasoning rather than merely teach rejection.

Cause-aware diagnosis separates missing information from constraint conflicts. Diagnostic tuning improves detection across both defect families. Qwen3-4B increases from 23.4% to 41.4% on

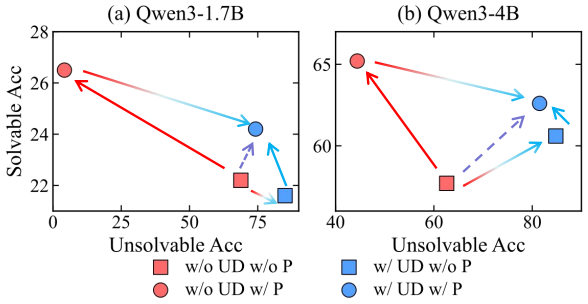


Figure 4: Visualizing the interaction between Solvable Accuracy (S) and Unsolvable Detection (U). The Purple Arrow highlights our full method (Blue Circle) achieving a Pareto improvement over the baseline (Red Square). Both models exhibit consistent dynamics: applying penalties without Unsolvable Data (UD) triggers **Capability Collapse** (Red Arrow), sacrificing U to maximize S. Conversely, when paired with UD, the penalty acts as a **Regularizer** (Blue Arrow), trading marginal U for significant gains in S compared to using UD alone (Blue Square).

AIME-MI and from 45.2% to 69.6% on AIME-CC. These gains indicate that unsolvability detection is not only a binary refusal skill, but can be decomposed into more specific answerability judgments.

Cause-aware diagnosis transfers to interactive clarification. To test whether boundary awareness leads to helpful behavior rather than over-refusal, we evaluate zero-shot transfer on the IN3 interactive clarification benchmark (Qian et al., 2024). Boundary-aware models ask for missing information more effectively: for Llama3-8B, Detail Recovery improves from 47.8% to 60.7%. This suggests that identifying the cause of underspecification can support proactive clarification in open-ended dialogue.

5.3 Ablation: Data–Penalty Interaction and Capability Collapse

To understand the contributions of data and reward mechanisms, we conducted a comprehensive ablation study comparing four configurations under the same 120-step training budget, defined by the inclusion of **Unsolvable Data (UnsData)** and the **False Unsolvability Penalty (P)**. Figure 4 compares: (1) **w/o UnsData w/o P** (baseline); (2) **w/o UnsData w/ P** (penalty only); (3) **w/ UnsData w/o P** (data only); and (4) **w/ UnsData w/ P** (our full method). The "Final" checkpoints in the main table are reported separately to show convergence, not used as baselines for these component-level ablations.

Explicit Unsolvable Data is a Prerequisite for Detection. As shown in Figure 4, the naive baseline trained without unsolvable data (*w/o UnsData*) lacks the fundamental capability to recognize unsolvability. For Qwen3-4B, detection is weak ($U = 62.6\%$). Introducing explicit unsolvable data provides the necessary supervision, boosting detection significantly ($U \rightarrow 84.8\%$). This confirms that optimizing solely for reasoning correctness on solvable tasks is insufficient.

Applying Penalty without Data Triggers Capability Collapse. Figure 4 (left) illustrates a critical failure mode caused by objective mismatch. When the penalty is enforced without providing ground-truth unsolvable examples, the model, lacking positive examples of valid refusal, minimizes the penalty risk by defaulting to answering all queries. This behavior effectively collapses its refusal mechanism, as seen in the 1.7B model's detection rate plummeting from 68.8% to 4.2%.

Applying Penalty with Data Facilitates a Strategic Trade-off. When grounded with adequate data, the penalty shifts from being a suppressor to a regularizer. For Qwen3-4B, adding the penalty (*w/ UnsData w/ P*) improves Solvable Accuracy (S) from 60.6% to 62.6%, at the cost of a slight drop in detection (U : $84.8\% \rightarrow 81.5\%$). This trade-off is strategic: it sacrifices marginal detection performance to enforce stricter reasoning rigor on feasible problems, optimizing the model for real-world utility.

Overall Gain: A Pareto Improvement over the Baseline. Viewing the trajectory globally (the purple dashed arrow in Figure 4b), our full method represents a comprehensive upgrade over the naive baseline (*w/o UnsData w/o P*). We achieve significant gains in both dimensions for Qwen3-4B (S : $57.7\% \rightarrow 62.6\%$, U : $62.6\% \rightarrow 81.5\%$). This confirms that UnsolvableRL does not merely shift the decision boundary, but fundamentally enhances both reliability and reasoning capability.

5.4 Calibration: Decoupling Objective Detection from Subjective Refusal

To test whether objective unsolvability detection must be separated from subjective capability calibration, we compare against a conflated-tag baseline and TruthRL (Wei et al., 2025). We also compare with a Fixed-τ variant to isolate the role of dynamic thresholding. Together, these baselines

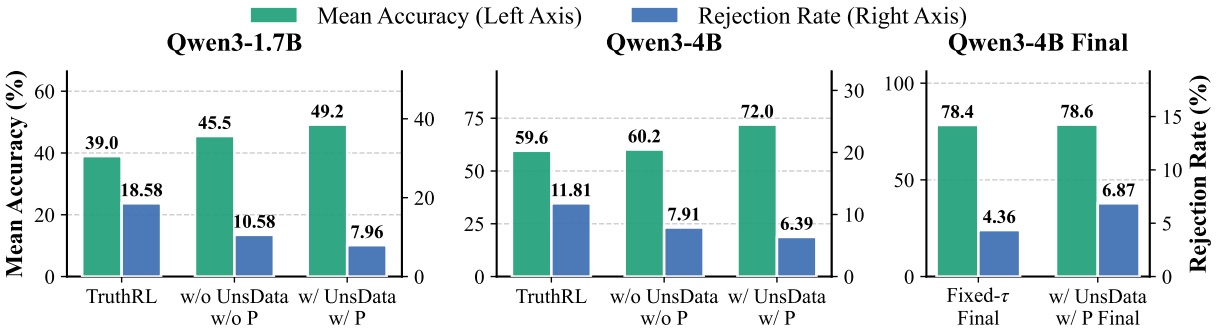


Figure 5: Performance Comparison and Calibration Analysis. (Left/Middle) Comparison with TruthRL. While TruthRL achieves higher rejection rates (Blue bars), it suffers from over-refusal, significantly degrading Accuracy (Green bars). (Right) Comparison of threshold strategies. The adaptive mechanism (*w/ P Final*) maintains a healthier rejection rate (6.87%) compared to the Fixed-τ baseline (4.36%) while preserving high accuracy.

test whether refusal should share one generic signal or be calibrated separately from input-side defect detection.

Necessity of Decoupling Objective and Subjective Refusal. We first investigated the necessity of decoupling objective detection from subjective calibration by training a baseline variant that conflates the two behaviors (i.e., the model outputs a unified `unsolvable` tag for any refusal, regardless of whether it stems from logical contradictions or subjective difficulty). We found that this undifferentiated approach results in significantly lower accuracy on solvable tasks across architectures. Specifically, after 120 training steps, the conflated model achieved a solvable set accuracy (S) of only 53.7% for Qwen3-4B and a mere 14.9% for Llama3-8B, compared to 62.6% and 40.1% achieved by our decoupled method at the same stage, respectively. This substantial performance gap, particularly evident in the Llama model, validates that distinguishing between "the problem is logically flawed" and "the problem exceeds my current capacity" is essential for preserving reasoning rigor; without this distinction, models tend to adopt a "lazy" refusal strategy to maximize rewards, severely discouraging attempts at complex reasoning. Detailed settings and **a diagnostic analysis** are provided in Appendix A.2.

Superior Balance Compared to TruthRL. We further benchmark our method against TruthRL (Wei et al., 2025) and different thresholding strategies. As shown in Left/Middle panels of Figure 5, TruthRL tends to be overly conservative, leading to over-refusal. For Qwen3-1.7B, while TruthRL achieves a high Rejection Rate (18.58%), its Accuracy drops to 39.0%, which is inferior to even the baseline. In contrast, our method achieves a significantly higher Accuracy (45.5%) with a moderate and precise Rejection Rate (10.58%). This indicates that our *target accuracy threshold* provides a softer, more distinctive signal than TruthRL's hard refusal reward, preserving the model's core reasoning capabilities.

Dynamic Thresholding Prevents Overconfidence. The rightmost panel of Figure 5 highlights the importance of our dynamic calibration. The Fixed-τ baseline yields a low rejection rate (4.36%), suggesting overconfidence. By dynamically adjusting τ based on the moving average of solvable confidence, our method (w/ P Final) maintains a robust rejection rate (6.87%) without sacrificing accuracy (78.6%), ensuring the model remains prudent even as it becomes stronger.

6 Conclusion

In this paper, we address the LLM "refusal dilemma" through solvability boundary learning, separating input-side answerability defects from model-side inability on well-posed tasks. Using our UnsolvableQA benchmark and UnsolvableRL framework, we demonstrate that recognizing missing-information and constraint-conflict defects acts as a powerful reasoning regularizer. This paradigm enables robust unsolvability detection ($U > 85\%$), sharper boundaries on solvable tasks, two-family defect diagnosis, and zero-shot transfer to interactive clarification; 4B/8B models further achieve OOD gains on GPQA (+12.0%) and ReliableMath (+12.1%). By turning refusal penalties from capability collapse into a rigor catalyst, our work distinguishes intrinsic query flaws from model limitations.

Limitations

Despite its diversity, our study focuses on structured missing-information and constraint-conflict defects, and does not cover the full spectrum of subjective ambiguity, undefined user intents, or subtle rule-level violations. Although diagnostic tuning shows zero-shot transfer to clarification behavior, our training objective remains primarily detection-oriented. A full closed-loop repair agent that optimizes when to ask, how to integrate user responses, and whether the repaired task is successfully solved remains future work.

References

Jiangjie Chen, Qianyu He, Siyu Yuan, Aili Chen, Zhicheng Cai, Weinan Dai, Hongli Yu, Qiying Yu, Xuefeng Li, Jiaze Chen, and 1 others. 2025a. Enigmata: Scaling logical reasoning in large language models with synthetic verifiable puzzles. *arXiv preprint arXiv:2505.19914*.

Qiguang Chen, Dengyun Peng, Jinhao Liu, HuiKang Su, Jiannan Guan, Libo Qin, and Wanxiang Che. 2025b. Aware first, think less: Dynamic boundary self-awareness drives extreme reasoning efficiency in large language models. *arXiv preprint arXiv:2508.11582*.

Qiguang Chen, Libo Qin, Jinhao Liu, Dengyun Peng, Jiannan Guan, Peng Wang, Mengkang Hu, Yuhang Zhou, Te Gao, and Wanxiang Che. 2025c. Towards reasoning era: A survey of long chain-of-thought for reasoning large language models. *arXiv preprint arXiv:2503.09567*.

Qiguang Chen, Libo Qin, Jiaqi Wang, Jinxuan Zhou, and Wanxiang Che. 2024. Unlocking the capabilities of thought: A reasoning boundary framework to quantify and optimize chain-of-thought. In *Advances in Neural Information Processing Systems*, volume 37, pages 54872–54904. Curran Associates, Inc.

Adam Fisch, Tommi Jaakkola, and Regina Barzilay. 2022. Calibrated selective classification. *arXiv preprint arXiv:2208.12084*.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Mengkang Hu, Yao Mu, Xinmiao Yu, Mingyu Ding, Shiguang Wu, Wenqi Shao, Qiguang Chen, Bin Wang, Yu Qiao, and Ping Luo. 2024. Tree-planner: Efficient close-loop task planning with large language models. In *International Conference on Representation Learning*, volume 2024, pages 48669–48696.

Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, and 1 others. 2024. Openai o1 system card. *arXiv preprint arXiv:2412.16720*.

Adam Tauman Kalai, Ofir Nachum, Santosh S. Vempala, and Edwin Zhang. 2025. Why language models hallucinate. *Preprint*, arXiv:2509.04664.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626.

Maor Juliet Lavi, Tova Milo, and Mor Geva. 2025. Detecting (un) answerability in large language models with linear directions. *arXiv preprint arXiv:2509.22449*.

Zhan Ling, Yunhao Fang, Xuanlin Li, Zhiao Huang, Mingu Lee, Roland Memisevic, and Hao Su. 2023. Deductive verification of chain-of-thought reasoning. *Advances in Neural Information Processing Systems*, 36:36407–36433.

Minghao Liu, Zonglin Di, Jiaheng Wei, Zhongruo Wang, Hengxiang Zhang, Ruixuan Xiao, Haoyu Wang, Jinlong Pang, Hao Chen, Ankit Shah, and 1 others. 2024. Automatic dataset construction (adc): Sample collection, data curation, and beyond. *arXiv preprint arXiv:2408.11338*.

Zhiting Mei, Christina Zhang, Tenny Yin, Justin Lidard, Ola Shorinwa, and Anirudha Majumdar. 2025. Reasoning about uncertainty: Do reasoning models know when they don't know? *arXiv preprint arXiv:2506.18183*.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. *Preprint*, arXiv:2203.02155.

Cheng Qian, Bingxiang He, Zhong Zhuang, Jia Deng, Yujia Qin, Xin Cong, Zhong Zhang, Jie Zhou, Yankai Lin, Zhiyuan Liu, and Maosong Sun. 2024. Tell me more! towards implicit user intention understanding of language model driven agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1088–1113, Bangkok, Thailand. Association for Computational Linguistics.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model. *Preprint*, arXiv:2305.18290.

Pranav Rajpurkar, Robin Jia, and Percy Liang. 2018. Know what you don't know: Unanswerable questions for squad. *Preprint*, arXiv:1806.03822.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *Preprint*, arXiv:1707.06347.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, and 1 others. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297.

Mark Steyvers, Heliodoro Tejeda, Aakriti Kumar, Catarina Belem, Sheer Karny, Xinyue Hu, Lukas W. Mayer, and Padhraic Smyth. 2025. What large language models know and what people think they know. *Nature Machine Intelligence*, 7(2):221–231.

Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, and 1 others. 2025. Kimi k1.5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*.

Songjun Tu, Jiahao Lin, Qichao Zhang, Xiangyu Tian, Linjing Li, Xiangyuan Lan, and Dongbin Zhao. 2025. Learning when to think: Shaping adaptive reasoning in r1-style models via multi-stage rl. *arXiv preprint arXiv:2505.10832*.

Jiakang Wang, Runze Liu, Lei Lin, Wenping Hu, Xiu Li, Fuzheng Zhang, Guorui Zhou, and Kun Gai. 2025. Aspo: Asymmetric importance sampling policy optimization. *Preprint*, arXiv:2510.06062.

Zige Wang, Wanjun Zhong, Yufei Wang, Qi Zhu, Fei Mi, Baojun Wang, Lifeng Shang, Xin Jiang, and Qun Liu. 2023. Data management for training large language models: A survey. *arXiv preprint arXiv:2312.01700*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.

Zhepei Wei, Xiao Yang, Kai Sun, Jiaqi Wang, Rulin Shao, Sean Chen, Mohammad Kachuee, Teja Gollapudi, Tony Liao, Nicolas Scheffer, and 1 others. 2025. Truthrl: Incentivizing truthful llms via reinforcement learning. *arXiv preprint arXiv:2509.25760*.

Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. 2023. Can llms express their uncertainty? an empirical evaluation of confidence elicitation in llms. *arXiv preprint arXiv:2306.13063*.

Boyang Xue, Qi Zhu, Rui Wang, Sheng Wang, Hongru Wang, Minda Hu, Fei Mi, Yasheng Wang, Lifeng Shang, Qun Liu, and Kam-Fai Wong. 2025. Reliablemath: Benchmark of reliable mathematical reasoning on large language models. *Preprint*, arXiv:2507.03133.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. *Preprint*, arXiv:2505.09388.

Dongkeun Yoon, Seungone Kim, Sohee Yang, Sunkyoung Kim, Soyeon Kim, Yongil Kim, Eunbi Choi, Yireun Kim, and Minjoon Seo. 2025. Reasoning models better express their confidence. *arXiv preprint arXiv:2505.14489*.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, and 16 others. 2025. Dapo: An open-source llm reinforcement learning system at scale. *Preprint*, arXiv:2503.14476.

Qingjie Zhang, Yujia Fu, Yang Wang, Liu Yan, Tao Wei, Ke Xu, Minlie Huang, and Han Qiu. 2025. On the self-awareness of large reasoning models' capability boundaries. *arXiv preprint arXiv:2509.24711*.

A Appendix

A.1 Comprehensive experimental setup and results

In this appendix, we provide the full, domain-level experimental results to supplement the aggregated metrics reported in the main text. Table 4 reports the performance breakdown across all seven datasets, including Game24, HamCycle, HamPath, Hitori, Maze-Easy, Maze-Hard, and AIME 24-25. We report three primary metrics to evaluate performance. First, Solvable Accuracy (S) measures the percentage of solvable problems correctly answered. Second, Unsolvable Detection Rate (U) indicates the percentage of unsolvable problems correctly identified as unsolvable. Finally, Mean (M) represents the arithmetic mean of S and U, reflecting the overall reliability of the model across different domains.

Complementing the standard metrics, Table 5 provides a diagnostic view focusing on the model's refusal behavior. Correctness (Corr) measures the accuracy of the model's responses on attempted questions, independent of the refusal decision. Rejection Rate (Rej) tracks the proportion of total queries, including both solvable and unsolvable instances, that the model refused to answer. The sum of these two metrics (C+R) serves as a diagnostic tool to analyze trade-offs between reasoning capability and refusal tendency, helping to identify issues such as over-refusal or capability collapse.

Both tables include results for a wide range of configurations. We compare against strong proprietary models like Deepseek-V3.2-R and Gemini-3.0-Pro, as well as the base Qwen3-Instruct models, which serve as reference points. To isolate the effects of our proposed components, we evaluate ablation variants trained for 120 steps under a fixed compute budget. These include models trained exclusively on solvable data without exposure to ground-truth unsolvable examples (denoted as w/o UnsData) and models trained without the False Unsolvability Penalty (denoted as w/o P), representing standard RLHF without the penalty term for incorrect refusals. Separately, we present fully converged checkpoints trained for longer budgets (e.g., 320 steps for Qwen3-4B), including a variant using a static confidence threshold (Fixed-tau Final) and our complete UnsolvableRL method (w/ UnsData w/ P Final). These final checkpoints are intended to show converged performance, whereas component-level conclusions are drawn from the same-budget 120-step ablations.

We also extended our evaluation to the Llama3-8B architecture to verify the generalizability of our method. As shown in the tables, training exclusively on solvable data (*w/o UnsData w/o P*) yields a dramatic improvement in solvable accuracy compared to the baseline ($S : 15.9\% \rightarrow 39.6\%$). However, this comes at a catastrophic cost: the model's ability to detect unsolvable problems collapses ($U : 34.6\% \rightarrow 4.1\%$), leading to a net decline in the overall Mean score ($M : 25.2\% \rightarrow 21.9\%$).

In contrast, our proposed method (*w/ UnsData w/ P*) successfully resolves this conflict. At step 120, it maintains superior solvable accuracy ($S = 40.1\%$) while simultaneously robustifying unsolvable detection ($U = 46.3\%$), nearly doubling the overall Mean score to 43.2%. **Furthermore, extending the training to approximately 400 steps (*Final*) demonstrates robust scalability: the model achieves further gains with Solvable Accuracy reaching 51.3% and Unsolvable Detection rising to 58.9%, resulting in a comprehensive Mean score of 55.1%.** This confirms that UnsolvableRL effectively prevents capability collapse and enhances reasoning rigor across different model families.

A.2 Analysis of the Conflated Refusal Baseline (+Mix Tag)

To further underscore the necessity of decoupling objective detection from subjective calibration, we evaluate a baseline variant referred to as *+Mix Tag*. This model is trained using the exact same dataset and hyperparameter configurations as our full *UnsolvableRL* framework. The critical distinction lies in the prompting strategy: the model is instructed to output a unified `unsolvable` tag for both objective contradiction detection and subjective difficulty-based refusal, rather than using distinct markers like `<beyond_capacity>`.

Under this conflated setting, the reward components defined in Section 4.3 are applied based on the ground truth of the instance, even though the model uses a single undifferentiated output:

- **Solvable Instances:** If the model emits the `unsolvable` tag, it inherently fails to solve the problem, resulting in $R_{\text{acc}} = 0$. However, to prevent the calibration mechanism from collapsing, the system treats this output as a subjective refusal and assigns the dynamic difficulty calibration reward R_{cal} according to

	Game24			HamCycle			HamPath			Hitori			Maze(E)			Maze(H)			AIME24-25			Overall		
Checkpoint	S	U	M	S	U	M	S	U	M	S	U	M	S	U	M	S	U	M	S	U	M	S	U	M
Deepseek-V3.2-R	98.0	77.5	87.8	83.5	94.0	88.8	84.0	96.0	90.0	70.0	86.0	78.0	100.0	98.9	99.5	98.0	98.0	98.0	85.0	40.3	62.7	88.4	84.4	86.5
Gemini-3.0-Pro	94.0	94.0	94.0	100.0	98.0	99.0	100.0	90.0	95.0	98.0	100.0	99.0	99.0	94.6	96.8	96.0	91.0	93.5	95.0	21.2	58.1	97.4	84.1	90.8
Qwen3-4B-Instruct	81.0	17.0	49.0	37.5	57.0	47.3	37.0	56.5	46.8	34.5	6.5	20.5	32.7	55.6	44.1	13.3	64.5	38.9	67.9	14.8	41.4	43.4	38.8	41.1
+ Mix Tag	82.0	~~92.0~~	87.0	31.3	~~100.0~~	66.0	38.0	~~100.0~~	69.0	48.0	~~90.0~~	69.0	85.0	~~91.5~~	88.2	33.0	~~84.0~~	58.5	58.3	~~61.3~~	59.8	53.7	~~88.4~~	~~71.1~~
+ TruthRL	91.0	91.0	91.0	32.3	86.0	59.2	49.0	90.0	69.5	38.0	15.0	26.5	90.0	77.1	83.6	31.0	56.5	43.8	67.5	19.1	43.3	57.0	62.1	59.6
+ w/o UnsData w/o P	83.0	92.0	87.5	36.5	81.0	58.8	47.0	84.0	65.5	46.0	20.0	33.0	88.5	76.6	82.6	34.5	60.5	47.5	68.3	24.0	46.2	57.7	62.6	60.2
+ w/o UnsData w/ P	90.5	12.5	51.5	39.6	67.5	53.5	49.5	64.5	57.0	56.5	4.0	30.3	94.8	83.2	89.0	56.3	72.0	64.1	69.0	7.4	38.2	65.2	44.4	54.8
+ w/ UnsData w/o P	84.0	95.0	89.5	39.6	96.0	67.8	53.0	100.0	76.5	45.0	87.0	66.0	95.5	95.2	95.4	40.0	89.5	64.8	67.1	30.8	49.0	60.6	84.8	72.7
+ w/ UnsData w/ P	88.0	97.0	92.5	35.9	95.5	65.7	51.5	99.5	75.5	48.5	71.0	59.8	94.0	95.5	94.8	53.3	79.8	66.6	66.9	31.9	49.4	62.6	81.5	72.0
+ Fixed-τ Final	87.5	99.0	93.3	39.1	100.0	69.6	67.0	99.5	83.3	51.0	98.0	74.5	96.3	100.0	98.2	52.5	95.8	74.2	69.0	41.9	55.5	66.1	90.6	78.4
+ w/ UnsData w/ P Final	94.5	99.0	96.8	41.1	94.5	67.8	55.0	96.5	75.8	63.5	94.5	79.0	96.5	98.9	97.7	65.8	89.0	77.9	69.6	40.4	55.0	69.4	87.5	78.6
Qwen3-1.7B-Instruct	78.0	23.0	50.5	22.9	28.0	25.5	14.0	33.0	23.5	8.0	21.0	14.5	0.0	85.6	42.8	0.0	80.0	40.0	38.3	21.2	29.8	23.0	41.7	32.4
+ TruthRL	76.0	80.0	78.0	19.8	49.0	34.4	17.0	58.0	37.5	4.0	25.0	14.5	1.5	82.4	42.0	0.0	73.5	36.8	42.1	17.0	29.6	22.9	55.0	39.0
+ w/o UnsData w/o P	77.0	88.0	82.5	25.0	60.0	42.5	15.0	83.0	49.0	5.0	53.0	29.0	0.5	93.6	47.1	0.0	79.5	39.8	32.9	24.4	28.7	22.2	68.8	45.5
+ w/o UnsData w/ P	85.0	2.0	43.5	22.4	10.5	16.5	25.5	14.0	19.8	12.0	2.5	7.3	0.0	0.0	0.0	0.0	0.0	0.0	40.8	0.5	20.7	26.5	4.2	15.4
+ w/ UnsData w/o P	70.0	97.0	83.5	22.9	94.0	58.5	22.0	97.0	59.5	4.0	79.0	41.5	0.5	95.2	47.9	0.0	86.0	43.0	31.7	46.8	39.3	21.6	85.0	53.3
+ w/ UnsData w/ P	82.0	96.5	89.3	18.2	85.0	51.6	20.5	89.5	55.0	8.0	62.0	35.0	0.0	90.7	45.4	0.0	74.3	37.2	40.8	21.2	31.0	24.2	74.2	49.2
+ w/ UnsData w/ P Final	84.0	100.0	92.0	20.8	88.0	54.4	27.0	85.0	56.0	11.0	59.0	35.0	0.0	95.2	47.6	0.0	79.0	39.5	35.4	28.7	32.1	25.5	76.4	50.9
Llama3-8B-Distill	28.0	48.0	38.0	27.1	2.0	14.6	14.0	0.0	7.0	6.0	56.0	31.0	1.0	44.7	22.9	0.0	52.0	26.0	35.0	39.4	37.2	15.9	34.6	25.2
+ Mix Tag	2.0	~~96.0~~	49.0	18.8	~~98.0~~	58.4	22.0	~~100.0~~	61.0	2.0	~~100.0~~	51.0	23.0	~~78.7~~	50.8	1.0	~~85.0~~	43.0	35.8	~~80.8~~	58.3	14.9	~~91.2~~	~~53.1~~
+ w/o UnsData w/o P	58.0	2.0	30.0	41.7	0.0	20.8	50.0	0.0	25.0	6.0	2.0	4.0	67.0	7.4	37.2	8.0	8.0	8.0	46.7	9.6	28.1	39.6	4.1	21.9
+ w/ UnsData w/ P	66.0	4.0	35.0	39.6	98.0	68.8	48.0	90.0	69.0	10.0	78.0	44.0	60.0	3.2	31.6	7.0	20.0	13.5	50.0	30.9	40.5	40.1	46.3	43.2
+ w/ UnsData w/ P Final	80.0	22.0	51.0	52.1	100.0	76.1	74.0	92.0	83.0	30.0	76.0	53.0	67.0	42.6	54.8	13.0	35.0	24.0	43.3	44.7	44.0	51.3	58.9	55.1

Table 4: Detailed performance breakdown (S/U/M) across all seven datasets. This table lists the full experimental results for all baselines and Qwen3 variants (including ablation checkpoints and final converged models). S: Solvable Accuracy; U: Unsolvable Detection Rate; M: Mean Score.

	Game24			HamCycle			HamPath			Hitori			Maze(E)			Maze(H)			AIME24-25			Overall		
Checkpoint	Corr	Rej	C+R	Corr	Rej	C+R	Corr	Rej	C+R	Corr	Rej	C+R	Corr	Rej	C+R	Corr	Rej	C+R	Corr	Rej	C+R	Corr	Rej	C+R
Deepseek-V3.2-R	87.8	0.00	87.8	88.8	8.30	97.1	90.0	6.00	96.0	78.0	9.00	87.0	99.5	0.00	99.5	98.0	0.00	98.0	62.7	0.80	63.5	86.5	3.44	90.0
Gemini-3.0-Pro	94.0	1.00	95.0	99.0	0.00	99.0	95.0	0.00	95.0	99.0	0.00	99.0	96.8	0.00	96.8	93.5	0.00	93.5	58.1	0.00	58.1	90.8	0.14	90.9
Qwen3-4B-Instruct	49.0	0.00	49.0	47.3	24.74	72.0	46.8	12.00	58.8	20.5	4.00	24.5	44.1	0.00	44.1	38.9	0.25	39.2	41.4	0.00	41.4	41.1	5.86	47.0
+ TruthRL	91.0	0.50	91.5	59.2	34.15	93.4	69.5	16.00	85.5	26.5	18.50	45.0	83.6	2.50	86.1	43.8	11.00	54.8	43.3	0.00	43.3	59.6	11.81	71.4
+ w/o UnsData w/o P	87.5	0.00	87.5	58.8	28.85	87.7	65.5	15.50	81.0	33.0	11.00	44.0	82.6	0.00	82.6	47.5	0.00	47.5	46.2	0.00	46.2	60.2	7.91	68.1
+ w/o UnsData w/ P	51.5	0.00	51.5	53.5	21.09	74.6	57.0	7.00	64.0	30.3	2.25	32.6	89.0	0.00	89.0	64.1	0.13	64.2	38.2	0.00	38.2	54.8	4.35	59.2
+ w/ UnsData w/o P	89.5	1.50	91.0	67.8	19.00	86.8	76.5	10.00	86.5	66.0	13.00	79.0	95.4	1.00	96.4	64.8	4.00	68.8	49.0	0.42	49.4	72.7	7.00	79.7
+ w/ UnsData w/ P	92.5	1.00	93.5	65.7	19.75	85.5	75.5	15.50	91.0	59.8	7.50	67.3	94.8	0.00	94.8	66.6	1.00	67.6	49.4	0.00	49.4	72.0	6.39	78.4
+ Fixed-τ Final	93.3	0.00	93.3	69.6	18.49	88.1	83.3	12.00	95.3	74.5	0.00	74.5	98.2	0.00	98.2	74.2	0.00	74.2	55.5	0.00	55.5	78.4	4.36	82.8
+ w/ UnsData w/ P Final	96.8	0.00	96.8	67.8	26.82	94.6	75.8	19.00	94.8	79.0	2.25	81.3	97.7	0.00	97.7	77.1	0.00	77.1	55.0	0.00	55.0	78.6	6.87	85.5
Qwen3-1.7B-Instruct	50.5	0.00	50.5	25.5	8.85	34.3	23.5	12.00	35.5	14.5	0.50	15.0	42.8	0.00	42.8	40.0	0.00	40.0	29.8	0.00	29.8	32.4	3.05	35.5
+ TruthRL	78.0	2.00	80.0	34.4	29.58	64.0	37.5	39.50	77.0	14.5	16.50	31.0	42.0	12.78	54.8	36.8	24.00	60.8	29.6	5.68	35.3	39.0	18.58	57.6
+ w/o UnsData w/o P	82.5	0.50	83.0	42.5	29.17	71.7	49.0	30.00	79.0	29.0	6.50	35.5	47.1	0.25	47.3	39.8	3.50	43.3	28.7	4.17	32.9	45.5	10.58	56.1
+ w/o UnsData w/ P	43.5	0.25	43.8	16.5	19.01	35.5	19.8	8.00	27.8	7.3	5.25	12.5	0.0	0.00	0.0	0.0	0.00	0.0	20.7	0.00	20.7	15.4	4.64	20.0
+ w/ UnsData w/o P	83.5	1.00	84.5	58.5	22.88	81.4	59.5	19.50	79.0	41.5	2.50	44.0	47.9	1.50	49.4	43.0	0.75	43.8	39.3	2.52	41.8	53.3	7.24	60.5
+ w/ UnsData w/ P	89.3	0.00	89.3	51.6	27.77	79.4	55.0	19.50	74.5	35.0	2.50	37.5	45.4	0.00	45.4	37.2	4.50	41.7	31.0	1.46	32.5	49.2	7.96	57.2
+ w/ UnsData w/ P Final	92.0	0.50	92.5	54.4	32.19	86.6	56.0	21.50	77.5	35.0	2.00	37.0	47.6	0.55	48.2	39.5	6.75	46.3	32.1	3.17	35.3	50.9	9.52	60.4
Llama3-8B-Distill	38.0	22.00	60.0	14.6	43.75	58.4	7.0	34.00	41.0	31.0	56.00	87.0	22.9	34.19	57.1	26.0	44.50	70.5	37.2	6.78	44.0	25.2	34.46	59.7
+ w/o UnsData w/o P	30.0	0.00	30.0	20.8	5.21	26.0	25.0	4.00	29.0	4.0	6.00	10.0	37.2	1.00	38.2	8.0	4.00	12.0	28.1	0.83	28.9	21.9	3.01	24.9
+ w/ UnsData w/ P	35.0	0.00	35.0	68.8	16.67	85.5	69.0	3.00	72.0	44.0	19.00	63.0	31.6	3.00	34.6	13.5	8.50	22.0	40.5	0.83	41.3	43.2	7.29	50.5
+ w/ UnsData w/ P Final	51.0	2.00	53.0	76.1	10.42	86.5	83.0	4.00	87.0	53.0	19.00	72.0	54.8	4.05	58.9	24.0	17.00	41.0	44.0	3.34	47.3	55.1	8.54	63.7

Table 5: Alternative metric analysis showing Correctness (**Corr**), Rejection Rate (**Rej**), and their sum (**C+R**). This breakdown distinguishes between general accuracy and refusal tendency, providing a diagnostic view for over-refusal and capability collapse.

Eq. 2.

- **Unsolvable Instances:** If the model emits the same `unsolvable` tag, it is rewarded with the detection signal $R_{\text{detect}} = +1$ (as detailed in Section 4.3), as the unified tag happens to match the objective ground truth of the problem statement.

As shown in the *+Mix Tag* row of Table 4, this setup creates a "reward shortcut" where the model is incentivized to emit the unified tag to hedge against two different reward sources. Notably, **after an identical duration of 120 training steps**, this confusion severely suppresses the model's incentive to attempt reasoning across different architectures. Consequently, within this **fixed budget of 120 steps**, the accuracy on solvable problems (S) reaches only 53.7% for Qwen3-4B and a mere 14.9% for Llama3-8B. Compared to the 62.6% (Qwen) and 40.1% (Llama) achieved by our decoupled method at the same training stage, the conflated approach results in a catastrophic reasoning deficit, particularly evident in the Llama model.

Explanation of Strikethroughs in Table 4. We have applied strikethroughs to the Unsolvable Detection Rate (U) and the resulting Mean Score (M) for the *+Mix Tag* row. These metrics are confounded and analytically meaningless in this context because the `unsolvable` tag no longer functions as a specific indicator of logical contradiction. In this undifferentiated regime, the high U scores (e.g., 91.2% for Llama3-8B and 88.4% for Qwen3-4B) do not represent superior logical rigor,

but rather a mixture of valid detection and "lazy" refusal to avoid potential penalties. Furthermore, this conflation diminishes real-world utility: users cannot discern whether the model refused because the query was fundamentally flawed or because it simply lacked the capacity to solve it.

A.3 Calibration Analysis

A.4 Details of Diagnostic Dataset Construction

To support the fine-grained diagnostic experiments described in Section 5.2, we curated a multi-domain dataset designed to evaluate a model's capability to categorize reasoning failures into two diagnostic families: **Missing Information (MI)**, which denotes missing support needed for a determinate answer under the requested response format; and **Constraint Conflict (CC)**, which introduces mutually incompatible conditions within a problem statement.

The diagnostic framework utilizes a rigorous split between training and evaluation to assess both cross-domain and temporal generalization. The training set comprises 259 instances, consisting of 140 multiple-choice problems from the MMLU-Pro validation seeds (70 solvable and 70 unsolvable) and 119 mathematical reasoning tasks derived from AIME 1983 (50 solvable and 69 unsolvable MI/CC instances). To establish a robust evaluation of out-of-distribution (OOD) performance, we constructed a test set of 320 instances using seeds entirely distinct from the training data. This includes 149 instances from the 2024–2025 AIME partitions (60 solvable and 89 unsolvable MI/CC instances) to test temporal generalization and 171 instances from GPQA to evaluate cross-domain transferability. The comprehensive breakdown of these datasets, including solvable and unsolvable distributions, is summarized in Table 6.

Split	Source	Solv.	Unsolv.	Total
Train	MMLU-Pro	70	70	140
	AIME (1983)	50	69	119
	Train Total	*120*	*139*	*259*
OOD Test	AIME (24–25)	60	89	149
	GPQA	100	71	171
	Test Total	*160*	*160*	*320*

Table 6: Detailed dataset statistics for fine-grained diagnostic reasoning.

A.5 Benchmarking SOTA Models on Diagnostic Reasoning

To establish a rigorous baseline and demonstrate the inherent difficulty of fine-grained unsolvability diagnosis, we benchmarked state-of-the-art (SOTA) large language models alongside our aligned open-source models on two Out-of-Distribution (OOD) test sets. The first is the 89 unsolvable MI/CC instances from our AIME (2024–2025) set (47 MI and 42 CC). The second is the 71 unsolvable instances from our GPQA dataset (GPQA-U).

We evaluated DeepSeek, Gemini, and GPT-5.2, alongside the baseline and diagnostic-tuned versions of Qwen3-4B and Llama3-8B. The metric of interest is *Diagnostic Accuracy* (the ability to correctly identify the specific root cause of unsolvability or explicitly reject the flawed query). The comprehensive results are presented in Table 7.

The empirical results reveal several critical insights into current LLM capabilities and the efficacy of our method:

First, **diagnostic tuning enables small models to rival giants**. Remarkably, explicit diagnostic alignment enables small open-source models to match or exceed massive proprietary systems on targeted tasks. For instance, Llama3-8B (+Diag.) achieves a 75.6% detection rate on AIME Constraint Conflicts (CC), surpassing DeepSeek (71.4%) and matching GPT-5.2 (76.2%). Furthermore, on the graduate-level GPQA-U set, our tuned Llama3-8B (36.6%) outperforms all evaluated SOTA models.

Second, there is a **stark variance across defect families in SOTA models**. While models like GPT-5.2 and DeepSeek perform reasonably well on explicit Constraint Conflicts ($> 70\%$ accuracy), their performance drops to around 50% for Missing Information. This suggests that current proprietary models rely heavily on surface-level logical conflicts (e.g., mutually exclusive rules) but struggle to independently verify whether a problem provides sufficient parameters for a determinate answer. This capability gap underscores the necessity of explicit diagnostic alignment frameworks like UnsolvableRL.

A.6 Out-of-Distribution (OOD) Generalization

To evaluate the generalization capabilities of UnsolRL, we utilized two Out-of-Distribution (OOD) benchmarks: **8-Puzzle** ($N_{sol} = 87, N_{unsol} = 100$)

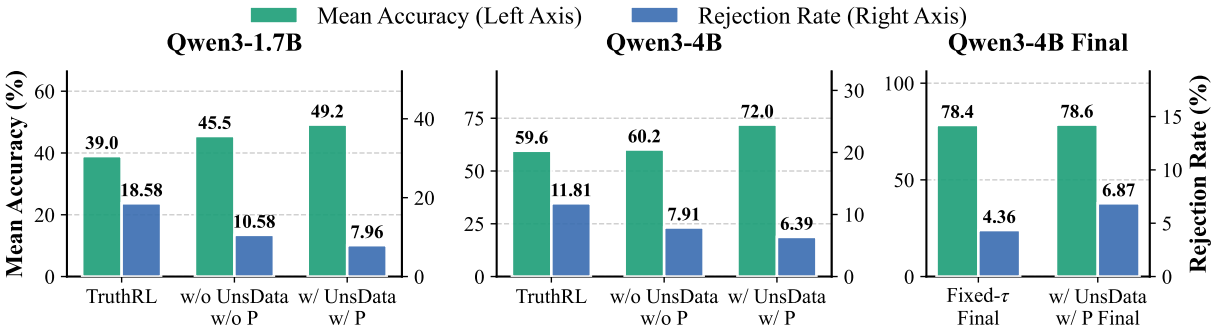


Figure 6: Performance Comparison and Calibration Analysis. (Left/Middle) Comparison with TruthRL. While TruthRL achieves higher rejection rates (Blue bars), it suffers from over-refusal, significantly degrading Accuracy (Green bars). (Right) Comparison of threshold strategies. The adaptive mechanism (*w/ P Final*) maintains a healthier rejection rate (6.87%) compared to the Fixed-τ baseline (4.36%) while preserving high accuracy.

Dataset Subset	**Aligned Open-Source Models**				**Proprietary SOTA Models**		
	Qwen3-4B		**Llama3-8B**		**DeepSeek**	**Gemini**	**GPT-5.2**
	Instruct	+ Diag.	Distill	+ Diag.			
AIME (2024–2025) Defect Families							
Missing Information (MI)	23.4	41.4	38.3	38.8	51.1	46.8	**55.3**
Constraint Conflict (CC)	45.2	69.6	45.2	**75.6**	71.4	47.6	**76.2**
General Science Defects							
GPQA-U	21.7	33.8	21.5	**36.6**	29.6	23.9	28.2

Table 7: Comprehensive diagnostic performance (%) across all evaluated models on OOD unsolvable instances. The table juxtaposes our aligned open-source models (4B/8B) against massive proprietary SOTA models. Note how diagnostic tuning (+*Diag.*) enables smaller models to bridge the gap and occasionally surpass SOTA baselines (e.g., Llama3-8B on CC and GPQA-U).

and **Zebra Logic** ($N_{sol} = 100, N_{unsol} = 100$). These tasks differ significantly from the training distribution, serving as a rigorous test for reasoning robustness. Table 8 details the performance mean@4 across Solvable (**S**), Unsolvable (**U**), and Mean (**M**) metrics.

SOTA Model Performance. We first established a baseline using top-tier proprietary models. **Deepseek-V3.2-R** achieves the highest overall performance (**M**=76.2%), demonstrating exceptional dominance in logic tasks with a mean score of 92.7% on Zebra Logic. In contrast, **Gemini-3.0-Pro** excels in procedural planning, securing the top spot on the 8-Puzzle benchmark (**M**=70.3%). **GPT-5.2-Low** shows balanced capabilities, particularly in detecting unsolvable instances across both domains (Overall **U**=94.3%). These results highlight that while leading models perform well, handling both logical constraints and path-finding robustly remains a non-trivial challenge.

Qwen3-4B Results. The baseline Qwen3-4B model struggles significantly with OOD detection, failing completely to identify unsolvable 8-Puzzle instances (0.0%). **UnsolRL-Final** substantially enhances robustness: it raises the **U** score to 15.8% on the 8-Puzzle and achieves a remarkable improvement on Zebra Logic, boosting **U** from 69.0% to 87.0%. Consequently, the Overall Mean (**M**) increases by over 50% relative to the baseline (from 23.2% to 34.9%), demonstrating that our method successfully transfers verification capabilities to unseen tasks.

Llama3-8B Results. Similarly, **Llama3-8B** exhibits substantial gains following UnsolRL alignment. The baseline model shows nearly no competence on the 8-Puzzle ($M = 1.0\%$), whereas the aligned version achieves a ten-fold improvement in mean performance (10.1%). On Zebra Logic, it maintains a robust unsolvable detection rate (78.0%) while doubling its solving accuracy ($1.5\% \rightarrow 4.0\%$). Overall, Llama3-8B's mean OOD score rises from 20.4% to 25.6%, confirming

the cross-architecture efficacy of our reinforcement learning framework.

Qwen3-1.7B Results. Despite its limited capacity, the 1.7B model exhibits a similar positive trend in anomaly detection. On Zebra Logic, UnsolRL improves the unsolvable detection rate (**U**) from 36.0% to 47.3%. Although the complex 8-Puzzle remains highly challenging for this smaller architecture (Overall **M** for 8-Puzzle improves marginally to 1.1%), the improvement in the Overall Mean across tasks (from 11.5% to 13.8%) confirms that UnsolRL effectively mitigates hallucinations on unsolvable queries, even when reasoning resources are constrained.

A.7 Case Study

To qualitatively demonstrate the effectiveness of our alignment strategy, we present three representative **Qwen3-4B** outputs from our converged UnsolvableRL checkpoint (320 steps, think mode). These examples illustrate the model's ability to navigate complex reasoning paths: detecting latent contradictions in mathematics (Case 1), identifying topological impossibility in logic puzzles (Case 2), and maintaining flexible reasoning for solvable problems (Case 3).

Case 1: Latent Unsolvability (Number Theory)

Problem Input. Let N be the greatest four-digit positive integer with the property that whenever one of its digits is changed to 1, the resulting number is divisible by 7. Suppose that $N \equiv Q + R \pmod 7$, where Q and R are the quotient and remainder when N is divided by 1000. Find $Q + R$.

Qwen3-4B Response.

> *"The digit-change property requires that for any digit of N, replacing that digit with 1 results in a number divisible by 7. Under the strict interpretation... the only solutions are $N = 5624$ and $N = 5694$, with the latter being the greatest. However, for $N = 5694$, computing $Q = 5$ and $R = 694$ gives $Q + R = 699$. Checking the congruence $N \equiv Q + R \pmod 7$ yields $5694 \equiv 3 \pmod 7$ and $699 \equiv 6 \pmod 7$, which are not equal. ... Thus, the greatest N satisfying the digit-change property does not satisfy the additional condition. Therefore, no four-digit N meets all the requirements."*

unsolvable

Reasoning Analysis. This case exemplifies **"Verify-then-Conclude"**. The contradiction is not explicit in the problem statement but lies deep within the solution space.

- **Deep Execution:** The model does not reject the problem early. It successfully solves the core number theory challenge, identifying the unique candidate $N = 5694$.

- **Constraint Verification:** Instead of hallucinating a match, it rigorously computes the modular arithmetic ($3 \neq 6$) and identifies the conflict between the derived candidate and the final constraint.

Case 2: Topological Impossibility (Hitori)

Problem Input. Solve the following 4×4 Hitori puzzle. Rules: No duplicates in rows/cols; Black cells cannot touch; White cells must be connected.

4	2	**3**	**3**
2	1	4	**2**
4	3	3	2
4	1	2	**2**

Qwen3-4B Response.

> *"The puzzle is unsolvable due to the isolation of cell (0,3) in all valid configurations... Specifically, the triplet of 2s in column 3 forces the outer cells (1,3) and (3,3) to be black to avoid adjacent shading. This forces (0,3) to be white. However, row 0 has duplicate 3s at (0,2) and (0,3), forcing (0,2) to be black. This leaves the white cell (0,3) surrounded by black cells (0,2) and (1,3), disconnecting it from the rest of the board."*

unsolvable

Reasoning Analysis. This demonstrates the model's grasp of **Global Constraints**.

- **Logical Deduction:** The model correctly deduces that to resolve duplicates in Col 3 (values: 3,2,2,2) and Row 0 (values: 4,2,3,3), specific neighbors of the corner cell (0,3) must be shaded.

- **Connectivity Check:** It recognizes that satisfying local uniqueness constraints inevitably violates the global connectivity rule (isolating the white cell at 0,3). This shows the model isn't just counting numbers but understanding the topological structure of the puzzle.

Case 3: Flexible Reasoning (Game24)

Problem Input. Use the numbers $\{4, 4, 6, 12\}$ to compute 24.

Model	8-Puzzle			Zebra Logic			Overall		
	S	U	M	S	U	M	S	U	M
Deepseek-V3.2-R	40.2	79.0	59.6	**94.5**	91.0	**92.7**	**67.3**	85.0	**76.2**
Gemini-3.0-Pro	**43.6**	**97.0**	**70.3**	65.0	74.0	69.5	54.3	85.5	69.9
GPT-5.2-Low	40.8	96.0	68.4	70.0	**92.5**	81.3	55.4	**94.3**	74.8
Qwen3-4B-Instruct	21.6	0.0	10.8	2.0	69.0	35.5	11.8	34.5	23.2
+ UnsolRL-Final	**26.4**	**15.8**	**20.1**	**8.8**	**87.0**	**47.9**	**17.4**	**52.4**	**34.9**
Qwen3-1.7B-Instruct	0.8	0.0	0.4	**9.0**	36.0	22.5	**4.9**	18.0	11.5
+ UnsolRL-Final	**1.2**	**1.0**	**1.1**	5.3	**47.3**	**26.3**	3.3	**24.2**	**13.8**
Llama3-8B-Distill	1.1	1.0	1.0	1.5	78.0	39.8	1.3	39.5	20.4
+ UnsolRL-Final	**10.3**	**10.0**	**10.1**	**4.0**	78.0	**41.0**	**7.2**	**44.0**	**25.6**

Table 8: OOD performance. **S**: Solvable Accuracy, **U**: Unsolvable Detection Rate, **M**: Mean Accuracy (%).

Qwen3-4B Response.

> *"To find an arithmetic expression that equals 24... One straightforward approach is to obtain a 4 from the numbers 12, 4, and 4, and then multiply the result by 6. 1. Start with 12. Subtract 4:* $12 - 4 = 8$. *2. Subtract the second 4:* $8 - 4 = 4$. *3. Multiply the result by 6:* $4 \times 6 = 24$. *This gives the expression:* $(12 - 4 - 4) \times 6$."

$$(12 - 4 - 4) * 6$$

Reasoning Analysis.

- **No False Refusal:** Despite the rigorous training on unsolvable data, the model correctly identifies a valid path for a solvable query.
- **Flexibility:** The model finds a subtraction-based solution different from the standard division-based answer ($4 \times (12/(6 - 4))$), showing that the alignment process has not constrained its exploration capability.

Summary. Together, these cases validate that UnsolvableRL does not teach the model to rely on simple heuristics for rejection. The aligned Qwen3-4B performs **comprehensive reasoning**—whether calculating modular congruences, checking graph connectivity, or exploring arithmetic combinations—before making a solvability determination.

A.8 Theoretical Justification and Empirical Ablation for False Unsolvability Penalty

We explicitly motivate the penalty value $\rho = -0.5$ via a decision-theoretic analysis. Consider a model with an initial weak reasoning capability ϵ on solvable tasks (e.g., $\epsilon \approx 0.1$) and a posterior belief p that a given problem is unsolvable. The model decides between two actions based on their expected rewards:

(1) Attempting to Solve: The expected reward depends on the problem being solvable and the model answering correctly: $\mathbb{E}[\text{Attempt}] \approx (1-p) \cdot \epsilon$.

(2) Declaring Unsolvable: The expected reward is the weighted sum of a correct rejection and a false rejection penalty: $\mathbb{E}[\text{Reject}] = p \cdot 1 + (1-p) \cdot \rho$.

The model will only attempt to reason if $\mathbb{E}[\text{Attempt}] > \mathbb{E}[\text{Reject}]$. Solving this inequality yields a critical threshold for the belief p:

$$p < \frac{\epsilon - \rho}{1 + \epsilon - \rho}$$

If we set $\rho = 0$ (no penalty), the threshold collapses to $p < 0.09$, meaning the model refuses to answer unless it is over 91% sure the problem is solvable. This leads to a local optimum of *universal rejection*. By setting $\rho = -0.5$, we relax this threshold to $p < 0.375$. This "Threshold Expansion" effectively encourages the model to attempt reasoning even under moderate uncertainty, preventing the collapse of reasoning capabilities.

To empirically validate this theoretical derivation, we conducted an ablation study on the penalty magnitude using the Qwen3-4B model at 40 training steps, as detailed in Table 9. We compared the default $\rho = -0.5$ against stricter penalties ($\rho = -1.0$ and $\rho = -2.0$). The results indicate that while the method is generally robust, continuing to decrease ρ (making the penalty more severe) is counterproductive. Specifically, decreasing ρ to -2.0 causes the Unsolvable Detection Rate (Overall U) to drop significantly from 75.8% to 69.4%, as the model becomes overly hesitant to declare unsolvability. Crucially, this sacrifice in detection yields only a marginal gain in Solvable Accuracy (Overall S increases from 47.0% to 47.9%), resulting in a decline in the global Mean score from

	Game24			HamCycle			HamPath			Hitori			Maze(E)			Maze(H)			AIME24-25			Overall		
Checkpoint	S	U	M	S	U	M	S	U	M	S	U	M	S	U	M	S	U	M	S	U	M	S	U	M
4B UnsolvableRL (P=-0.5, S40)	79.0	93.0	86.0	29.2	98.0	63.6	48.0	98.0	73.0	37.0	41.0	39.0	53.0	91.0	72.0	13.0	78.0	45.5	70.0	31.9	51.0	47.0	75.8	61.4
4B UnsolvableRL (P=-1.0, S40)	74.0	90.0	82.0	35.4	98.0	66.7	34.0	98.0	66.0	50.0	36.0	43.0	48.0	74.5	61.2	22.0	78.0	50.0	68.3	29.7	49.0	47.4	72.0	59.7
4B UnsolvableRL (P=-2.0, S40)	80.0	90.0	85.0	29.2	98.0	63.6	40.0	98.0	69.0	42.0	22.0	32.0	61.0	70.7	65.8	15.5	76.5	46.0	67.5	29.7	48.6	47.9	69.4	58.6

Table 9: Detailed performance breakdown (S/U/M) across all seven datasets for different Penalty values at 40 steps.

Algorithm 1 Game24 Generation and Verification

Require: Difficulty level D (determines count $k \in \{4, 5, 6\}$), Target $T = 24$
Ensure: A paired instance (S, Label)

Function VERIFY(S):
  $\mathcal{T} \leftarrow$ All binary expression trees for set S
  for each tree $t \in \mathcal{T}$ **do**
    Evaluate $val \leftarrow t$ using **Rational Arithmetic**
    if $val == T$ **then return** SOLVABLE, t
  end for
  return UNSOLVABLE, $\emptyset$
End Function
repeat
  Sample a set of k integers $S \leftarrow \{n_1, \ldots, n_k\}$ where $n_i \in [1, 13]$
  $Status, Proof \leftarrow$ VERIFY(S)
  if goal is *Solvable* **and** $Status ==$ SOLVABLE:
    return $(S, Proof)$
  else if goal is *Unsolvable* **and** $Status ==$ UNSOLVABLE:
    return $(S,$ "Unsolvable")
until Valid instance found

61.4% to 58.6%. Consequently, $\rho = -0.5$ proves to be the optimal value, striking the best balance between maintaining reasoning rigor and preserving the courage to reject unsolvable queries. To ensure reliability, we report the average results across two independent training runs, evaluated using mean@2.

A.9 Theoretical Justification and Empirical Ablation for Capability-Refusal Dynamics

In this section, we derive the mathematical relationship between the model's reasoning capability and its tendency to refuse, explaining why a dynamic threshold is essential.

The model should choose to refuse only if the reward for refusal exceeds the expected reward of answering:

$$R_{\text{cal}} > \mathbb{E}[R_{\text{ans}}]. \tag{3}$$

Note that the expected reward for answering is always non-negative ($\mathbb{E}[R_{\text{ans}}] \geq 0$).

Failure of Fixed Threshold. If we were to use a *fixed threshold* (e.g., $\tau = 0.5$), as the model's reasoning capability improves during training, the batch accuracy β will likely surpass τ (i.e., $\beta > \tau$). Once this occurs, the refusal reward becomes **negative** ($R_{\text{cal}} = \lambda(\tau - \beta) < 0$).

Substituting this into Inequality 3, the condition becomes Negative Value $> \mathbb{E}[R_{\text{ans}}]$. Since $\mathbb{E}[R_{\text{ans}}] \geq 0$, this inequality is impossible to satisfy. Consequently, for well-performed batches, the model is incentivized to suppress refusal entirely to avoid penalties, effectively forcing the model to hallucinate answers on difficult problems rather than refusing.

Our progressive schedule prevents this collapse by ensuring τ grows dynamically to maintain $\tau \gtrsim \beta$, keeping the refusal incentive positive for uncertain queries. Specifically, in our experiments, the Fixed-τ baseline uses a constant $\tau = 0.4$, while our dynamic schedule linearly increases τ from 0.4 to 1.0 over 400 training steps.

To further validate our threshold selection, we conducted a simple ablation study on τ. As shown in Table 10, we found that when training for only 40 steps, setting $\tau = 0.4$ generally yields strong Accuracy, while further decreasing τ has minimal impact on performance. Based on these observations, we adopted $\tau = 0.4$ as the initial value for our main experiments.

A.10 Reverse Construction Pipeline for Non-Programmatic Reasoning

Unlike logic puzzles (e.g., Maze or Game24) that can be verified via deterministic algorithms, mathematical reasoning tasks like AIME require a model-based approach to introduce and verify contradictions. We propose a **Reverse Construction** pipeline that leverages the deductive capabilities of DeepSeek-Reasoner and Gemini to transform solvable seeds into logically flawed problems. The pipeline consists of seven stages, driven by meticulously designed prompts to ensure maximum logical rigor.

Step 1: Solvable Grounding. To ensure the

	Game24			HamCycle			HamPath			Hitori			Maze(E)			Maze(H)			AIME24-25			Overall		
Checkpoint	Corr	Rej	C+R	Corr	Rej	C+R	Corr	Rej	C+R	Corr	Rej	C+R	Corr	Rej	C+R	Corr	Rej	C+R	Corr	Rej	C+R	Corr	Rej	C+R
4B UnsolvableRL (τ=0.2, S40)	90.5	3.00	93.5	63.7	23.21	86.9	72.5	10.50	83.0	32.5	6.50	39.0	69.0	8.75	77.7	46.5	21.00	67.5	49.5	0.00	49.5	60.6	10.42	71.0
4B UnsolvableRL (τ=0.4, S40)	86.0	1.50	87.5	63.6	24.48	88.1	73.0	5.50	78.5	39.0	10.00	49.0	72.0	4.50	76.5	45.5	18.50	64.0	51.0	0.21	51.2	61.4	9.17	70.6
4B UnsolvableRL (τ=0.6, S40)	86.0	3.00	89.0	58.2	31.19	89.4	66.0	13.50	79.5	39.5	8.00	47.5	71.3	6.75	78.1	47.8	26.50	74.3	48.0	0.83	48.8	59.5	12.82	72.3
4B UnsolvableRL (τ=0.8, S40)	78.0	3.00	81.0	67.3	30.69	98.0	70.5	13.50	84.0	36.0	8.00	44.0	70.7	6.75	77.5	49.3	26.25	75.6	46.4	0.84	47.2	59.7	12.72	72.4
4B UnsolvableRL (τ=1.0, S40)	85.5	3.50	89.0	59.7	35.19	94.9	65.5	21.50	87.0	26.5	24.50	51.0	78.8	2.75	81.6	47.0	26.25	73.3	49.3	0.42	49.7	58.9	16.30	75.2

Table 10: Detailed performance breakdown (Correctness/Rejection Rate/C+R) across all seven datasets for different τ values at 40 steps.

construction is based on a valid foundation, a primary model (DeepSeek-Reasoner) solves a seed problem from AIME (1983–2023) or MMLU-Pro. The resulting reasoning chain (CoT_A) is extracted as a reference roadmap.

> ***Prompt (Base Verification):*** *You are a top logical reasoning expert. Solve the following problem. If possible, include a brief chain-of-thought and a clear final answer (boxed or labeled 'Answer:'). Treat the content between "— Problem Start —" and "— Problem End —" purely as the problem statement.*

Step 2: Strategic Contradiction Planning. The model acts as a "Senior Designer" to plan a contradiction based on CoT_A, explicitly instructing the model to target the *late-stage* of the reasoning to bypass simple heuristics. For main **UnsolvableQA** construction, we use constraint-conflict planning:

> ***Prompt (Constraint Conflict - CC):*** *Construct a subtle Constraint Conflict aimed to be hard to spot: prefer placing the contradictory condition near the END of the original chain-of-thought so it appears only after most reasoning steps. [...] Identify a late-stage step where a small external constraint can be added to produce a logical conflict.*

For the **diagnostic extension** (Section 5.2), we additionally use a missing-information planning prompt:

> ***Prompt (Missing Information - MI):*** *Design an UNSOLVABLE problem by REMOVING or OMITTING essential information. CRITICAL DISTINCTION: The problem has NO answer. [...] UNDERDETERMINED / MULTIPLE SOLUTIONS (what we do NOT want): The problem has infinitely many valid answers because one constraint was removed (e.g. one equation in two unknowns). Do NOT create such a problem.*

Step 3: Unsolvable Problem Synthesis. The model integrates the generated plan into a natural-sounding problem statement.

> ***Prompt (Generation):*** *Using the provided contradiction plan, produce a modified problem that embeds the contradiction in a way that is DETECTABLE by a rigorous solver, while appearing natural. The contradiction should manifest during problem-solving (preferably in later steps), but must be UNAVOIDABLE and CLEAR once the solver reaches that point. Output only the modified problem text followed by the single-line '#GEN_VERIFY:' comment.*

Step 4–5: Autonomous Solve-and-Detect Loop. To ensure objective detectability, a "blind" solver attempts to detect the flaw without context (Step 4). If the detection fails, a context-aware "Recheck" (Step 5) isolates whether the flaw is genuinely present but missed, or if the construction failed.

> ***Prompt (Step 4: Blind Detection):*** *Analyze the following problem VERY CAREFULLY, checking for contradictions at EVERY step. If you find ANY contradiction, conflict, or inconsistency (no matter how subtle), you MUST immediately stop and conclude the problem is unsolvable. Output EXACTLY \boxed{Unsolvable}.*

> ***Prompt (Step 5: Contextual Recheck):*** *You will receive: 1) ORIGINAL PROBLEM, 2) ORIGINAL CoT_A, 3) CONTRADICTION PLAN, 4) GENERATED UNSOLVABLE PROBLEM, 5) FIRST VERIFICATION ATTEMPT (CoT_B). [...] TASK: A) Does the generated problem actually contain a logical contradiction? B) If yes, did CoT_B miss the contradiction due to flawed reasoning, or was the problem construction itself flawed?*

Step 6–7: Cross-Model Consensus and Adjudication. To eliminate model-specific bias, Gemini 1.5 Pro independently audits the generated problem (Step 6). Disagreements trigger an iterative adjudication stage (Step 7) where models debate to reach a unanimous verdict.

> ***Prompt (Step 7: DeepSeek Adjudicates Gemini):*** *Another evaluator (Gemini) has concluded that a candidate "unsolvable" problem is actually SOLVABLE. We need your independent judgment. [...] TASK: Do you agree with Gemini that this problem is solvable (no real contradiction), or do you still believe the problem is unsolvable? Answer with clear reasoning.*

Formatting Suffix (Appended to all Evaluation Tasks): To standardize parser behavior during evaluation and RL training, all instances require a specific diagnostic output format:

> *If you determine that this problem is unsolvable, you must output exactly ONE of the following in*

\boxed{} at the end, according to the reason:
- \boxed{unsolvable_constraint_contradiction}
- \boxed{unsolvable_missing_information}
If the problem is solvable, output \boxed{your_numeric_answer} as usual.

Step 8: Expert Human Audit and Quality Control. To ensure absolute logical precision and eliminate any remaining artifacts from the automated generation process, all instances that pass the consensus phase undergo a final manual audit by human experts. We employ an audit team of professionals with backgrounds in STEM fields, all holding a Master's degree or higher, to conduct a granular, instance-by-instance verification. Reviewers are provided with the complete data package—comprising the original problem, the contradiction plan, and the rewritten version—to evaluate three core indicators: (1) *Mathematical Integrity*, ensuring the problem statement is clear, the notation is standardized, and the overall formulation is a well-posed mathematical statement; (2) *Modification Validity*, confirming that the rewritten version accurately executed the intended plan to alter key constraints; and (3) *Definitive Unsolvability*, verifying that the problem is demonstrably impossible to solve in a logical sense, rather than being merely ambiguous or under-determined due to missing information. Finally, we adopt a rigorous elimination mechanism where only instances that are found to be logically consistent and unambiguously unsolvable according to these criteria are retained in the final **UnsolvableQA** dataset. Throughout the screening process, we manually excluded 5 ambiguous instances from the newly constructed unsolvable test set, ensuring that all remaining entries meet our quality standards.

Case Study: Human-Led Detection of Geometric Redundancy

- **Original Problem:** Six points A, B, C, D, E, F lie on a line in that order. Given distances $AC = 26, BD = 22, CE = 31, DF = 33, AF = 73$, and external distances $CG = 40, DG = 30$, find the area of $\triangle BGE$.
- **Generator's Modification:** Added the constraint: "G lies to the left of point E."
- **Automated Pipeline Assumption:** Both the generator and the automated cross-model verifiers accepted this instance. They assumed a fixed coordinate system (e.g., A at $x = 0$) where G was calculated to be at $x \approx 58.5$ and E at $x = 57$. Since $58.5 > 57$, the automated tools concluded that the constraint "G is to the left of E" created a definitive logical contradiction.
- **Human Expert Discovery: Crucially, during the manual audit, our experts identified a subtle flaw missed by the models.** They recognized that the problem lacks an absolute coordinate orientation, granting it a "Mirror Symmetry" degree of freedom. By simply flipping the line's orientation (setting F at $x = 0$), G and E swap their relative positions while all distance constraints remain intact. In this mirrored state, G is indeed to the left of E, making the problem perfectly solvable.
- **Final Decision: Rejected by Human Audit.** This case highlights how human experts can detect "self-healing" geometric properties and coordinate redundancies that current LLMs fail to perceive, ensuring the dataset's labels remain incontestable.

A.11 Logic Puzzle Generation Algorithms

A.11.1 Game24 Generation

The Game24 task involves combining a set of integers using arithmetic operations $(+, -, \times, \div)$ to equal 24. To ensure absolute correctness, particularly for unsolvable instances, we employ an exhaustive search over all possible binary expression trees. Crucially, we utilize exact rational arithmetic (e.g., Python's `Fraction`) rather than floating-point calculations to prevent precision errors that could misclassify instances (e.g., $23.99999 \neq 24$).

For unsolvable instances, we use a rejection sampling strategy: we randomly sample integer sets and retain only those for which the exhaustive solver proves the solution set is empty.

A.11.2 Hamiltonian Cycle and Path Generation

The Hamiltonian Cycle problem asks whether a graph contains a closed loop visiting every vertex exactly once, while the Hamiltonian Path problem asks for a linear path visiting every vertex. Since these problems are NP-complete, we utilize a SAT-based verification pipeline to guarantee ground-truth labels.

SAT Encoding. We encode the existence of a path/cycle as a Boolean Satisfiability Problem. Let

N be the number of vertices. We introduce boolean variables $x_{v,i}$ indicating that vertex v occupies the i-th position in the sequence ($0 \leq i < N$). The constraints are defined as follows: **Bijection** requires that $\sum_v x_{v,i} = 1$ for all i and $\sum_i x_{v,i} = 1$ for all v. **Adjacency** mandates that for all $i \in [0, N-2]$, if $x_{u,i}$ and $x_{v,i+1}$ are true, then (u, v) must be an edge in E. Finally, **Cycle Closure (Cycle Only)** ensures that if $x_{u,N-1}$ and $x_{v,0}$ are true, then $(u, v) \in E$. We use the Minisat solver (via `pysat`) to determine satisfiability.

Unsolvability Injection. Instead of purely random sampling, we inject structural properties that inherently prevent Hamiltonian traversals. These include: **Structural Bottlenecks**, where graphs are generated with "bridge" nodes or articulation points that partition the graph into components that cannot be traversed in a single pass; **Degree Constraints**, where we inject nodes with degree < 2 (dead ends) for cycles, or ensure the presence of > 2 nodes with degree 1 for paths; and **Disconnectivity**, where we explicitly construct graphs with multiple connected components.

A.11.3 Hitori Generation

Hitori is a logic puzzle played on an $N \times N$ grid. The goal is to shade cells such that no number appears more than once in any row or column, shaded cells do not touch orthogonally, and all unshaded cells remain connected.

We model the puzzle as a Constraint Satisfaction Problem (CSP). We define binary variables $x_{i,j} \in \{0, 1\}$ for each cell (1 denotes shaded). The key constraints are: **Uniqueness**, which requires that if a number v appears k times in any row/column, at least $k - 1$ occurrences must be shaded; **Non-adjacency**, ensuring that for any adjacent cells u, v, $x_u + x_v \leq 1$; and **Connectivity**, verified via BFS to ensure all cells where $x_{i,j} = 0$ form a single connected component.

Solvable instances are filtered to ensure a *unique* solution. Unsolvable instances are those where the CSP solver (combined with connectivity checks) returns a solution count of 0.

A.11.4 Maze Navigation Generation

The Maze Navigation task requires finding a valid path from a start position (S) to an end position (E) in a grid-based maze, where movement is restricted by walls.

Algorithm 2 Hamiltonian Graph Generation

Require: Type $T \in \{\text{Cycle}, \text{Path}\}$, Node count N
Ensure: A graph $G = (V, E)$ with label
Function VERIFYSAT(G, T):
  Construct CNF formula ϕ based on adjacency matrix of G
  Add bijection constraints to ϕ
  if T == Cycle: Add closure constraint $v_{N-1} \leftrightarrow v_0$
  return SAT-SOLVER(ϕ)
End Function
repeat
  Initialize empty graph G on N nodes
  if goal is *Solvable*:
    Create a random permutation P of vertices
    Add edges (P_i, P_{i+1}) to form a base Path/Cycle
    Add random noise edges to increase density
  else if goal is *Unsolvable*:
    Select strategy $\sigma \in \{\text{Disconnect}, \text{Bottleneck}, \text{DeadEnd}\}$
    Construct G satisfying σ (e.g., partition V into V_1, V_2 with no edges between them)
    Add random noise edges strictly preserving σ
  $IsFeasible \leftarrow$ VERIFYSAT(G, T)
until ($IsFeasible$ matches goal)
return G

Verification Logic. We employ Breadth-First Search (BFS) to determine solvability. BFS guarantees finding the shortest path if one exists. A maze is **Solvable** if a path from S to E exists, and **Unsolvable** if the set of reachable cells from S does not contain E.

Construction Pipeline. We first generate a complex solvable maze using a randomized Depth-First Search (DFS) algorithm, which ensures a spanning tree structure. To increase complexity, we selectively remove walls to introduce cycles and multiple potential paths, while validating that the solution remains unique or manageable using BFS.

To construct **Unsolvable Instances**, we employ a *Strategic Blockage* method rather than random wall placement. We analyze the solvable maze to identify all valid paths from S to E, and then select a *Critical Junction*, a coordinate (x, y) that lies

Algorithm 3 Hitori Generation via CSP

Require: Grid size N
Ensure: A validated Hitori grid G
Function SOLVEHITORI(G):
  Initialize CSP solver P
  Apply **Row/Col constraints**: Eliminate duplicates
  Apply **Adjacency constraints**: No adjacent black cells
  $\mathcal{S}_{local} \leftarrow P$.getSolutions()
  $\mathcal{S}_{valid} \leftarrow \emptyset$
  for each solution $s \in \mathcal{S}_{local}$ **do**
    if CHECKCONNECTIVITY(s) **then** $\mathcal{S}_{valid}$.add(s)
  end for
  return $|\mathcal{S}_{valid}|$
End Function
repeat
  Generate random $N \times N$ grid G with values $\in [1, N]$
  $Count \leftarrow$ SOLVEHITORI(G)
  if goal is *Solvable* **and** $Count == 1$:
    return G {Unique solution found}
  else if goal is *Unsolvable* **and** $Count == 0$:
    return G {Provably no valid configuration}
until Valid instance found

Algorithm 4 Maze Generation with Strategic Blockage

Require: Dimensions W, H, Difficulty D
Ensure: A maze grid M and label L
Function VERIFY(M):
  return BFS(M, Start, End)
End Function
repeat
  $M_{base} \leftarrow$ GENERATESOLVABLEDFS(W, H)
  if goal is *Solvable*:
    return (M_{base}, VERIFY(M_{base}))
  else if goal is *Unsolvable*:
    $\mathcal{P} \leftarrow$ All paths in M_{base} from Start to End
    Select path $P \in \mathcal{P}$ based on difficulty D
    Select obstacle point $o \in P$ (excluding Start/End)
    $M_{blocked} \leftarrow M_{base} \cup \{o\}$
    if not VERIFY($M_{blocked}$):
      return ($M_{blocked}$, "Unsolvable")
until Valid instance found

on a valid path (or is shared by multiple paths). We place an obstacle (wall) at (x, y) and rigorously verify unsolvability using BFS. If the maze remains solvable (e.g., via a detour), we discard the instance or iteratively block remaining paths until connectivity is severed. This approach ensures that unsolvable mazes are structurally similar to solvable ones, often requiring the model to explore deep into the maze before realizing the path is blocked, thereby penalizing premature "give up" or hallucinated paths.

A.12 Hyperparameters and Training Details

We implement our training pipeline using the `verl` (HybridFlow) framework and vllm engine (Sheng et al., 2025; Kwon et al., 2023), which supports efficient reinforcement learning. The specific hyperparameters used for Group Relative Policy Optimization (GRPO) are detailed in Table 11. We set the group size G (denoted as `rollout.n` in `verl`) to 12. Notably, we explicitly set `use_kl_loss=False`, meaning the KL divergence penalty term was disabled during optimization, relying instead on the probability ratio clipping to maintain policy stability.

We differentiate the batch size for Qwen3-1.7B and Qwen3-4B to accommodate their respective memory footprints. Specifically, the 1.7B model utilizes a larger batch size (32×12) compared to the 4B model (14×12). This adjustment balances the computational overhead, resulting in comparable training speeds for both architectures; typically, completing 120 training steps requires approximately one day on our 8 $\times$ NVIDIA A100 GPU cluster. Constrained by GPU resources, we conducted a single training run for the main experiments and report results using mean@8 for AIME and mean@4 for the remaining benchmarks. Throughout the training process, we observed a stable increase in both Solvable Accuracy and Unsolvable Detection Rate.

To further analyze the training dynamics, Figure 7 illustrates the evolution of the average response token length throughout the training process. We observe a general upward trend for both models, indicating that the RL alignment encourages the generation of more comprehensive reasoning chains (longer CoT) to distinguish solvability. The Qwen3-4B model exhibits higher volatility (indicated by the wider shaded area) compared to

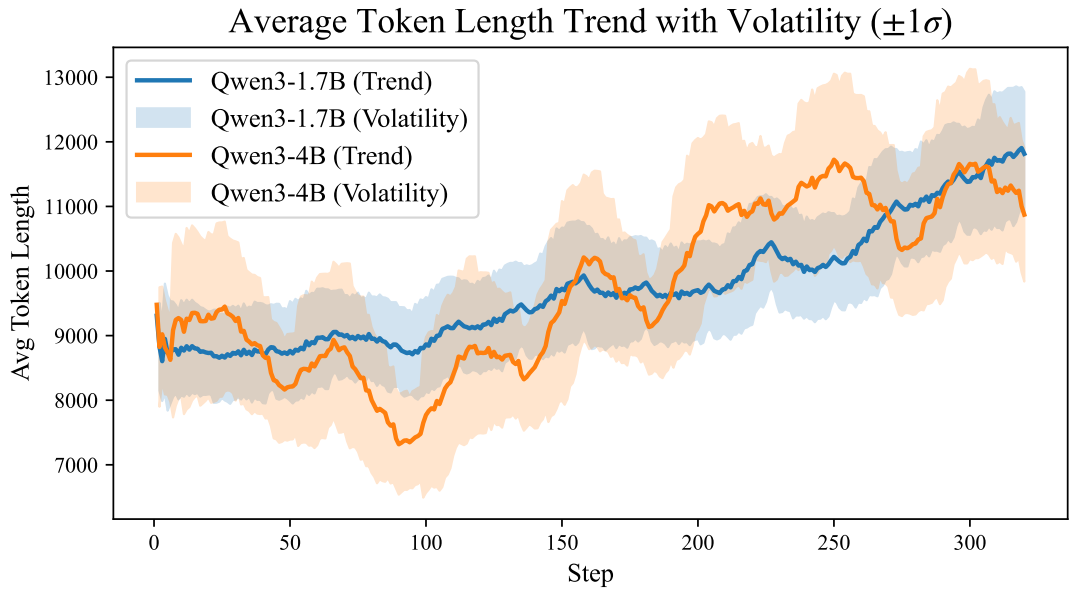


Figure 7: Average response token length trend during training. The solid lines represent the moving average, and the shaded regions indicate the volatility ($\pm 1\sigma$). Both models show a tendency to increase reasoning length as training progresses.

the 1.7B model, suggesting more aggressive exploration within the solution space.

To explicitly guide the model's decision-making, we append specific instructions to the system prompt. For **unsolvable detection**, the instruction is: "*If you believe the problem is unsolvable, please output* `\boxed{unsolvable}` *at the end.*" For **capability calibration** (refusal), the instruction is: "*If your chain of thought is still confused after prolonged reasoning and you are likely to get the answer wrong, please provide the final output* `\boxed{Beyond my capabilities}`.". We rely on these predefined output formats to programmatically extract and classify the model's final response.

We will release our dataset, data construction pipeline, and training code upon acceptance.

Table 11: Hyperparameter settings used in our experiments. We employ Group Relative Policy Optimization (GRPO) with the KL divergence penalty explicitly disabled to encourage broader exploration within the solution space.

Hyperparameter	**Value**
General Optimization & Generation	
RL Algorithm	GRPO
Group Size (G)	12
KL Penalty Term	Disabled
PPO Mini-Batch Size	8
Sampling Temperature	0.6
Sampling Top-k	45
Max Prompt Length	2048
Generation Engine	VLLM
Infrastructure	
Hardware	8 × NVIDIA A100 GPUs
Tensor Parallelism (TP)	2
Model-Specific Settings	
Qwen3-1.7B	
Training Batch Size	32×12
Max Response Length	32,000
Qwen3-4B	
Training Batch Size	14×12
Max Response Length	32,000